



AMERICAN
UNIVERSITY
OF BEIRUT

Fall 2025

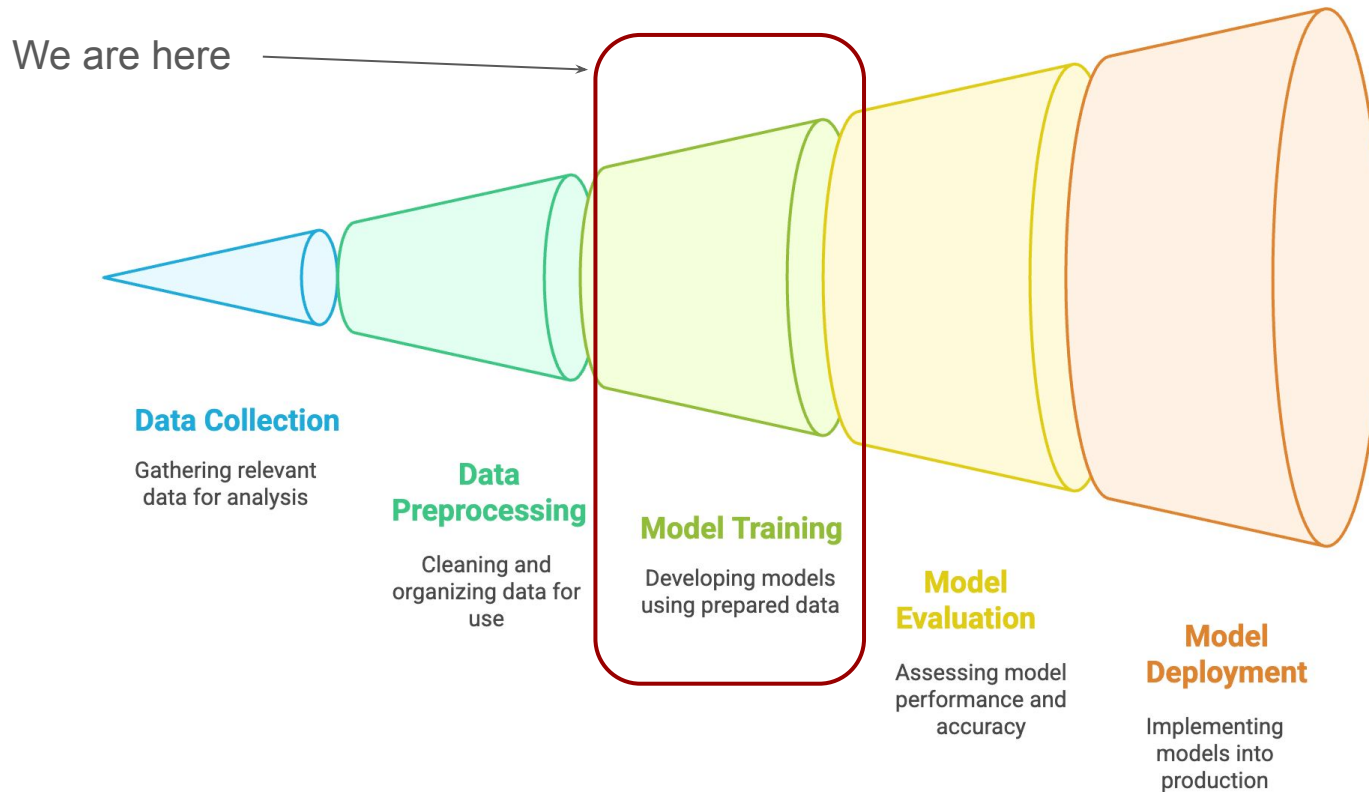
EECE 490: Introduction to Machine Learning

C4 - Supervised Machine Learning Algorithms



Machine Learning Lifecycle

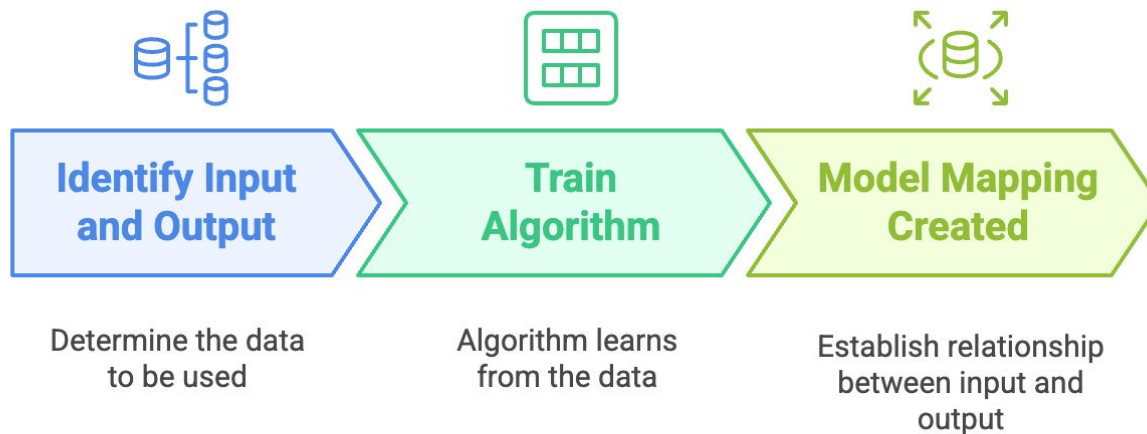
Where are we in the life cycle now?



Supervised Learning

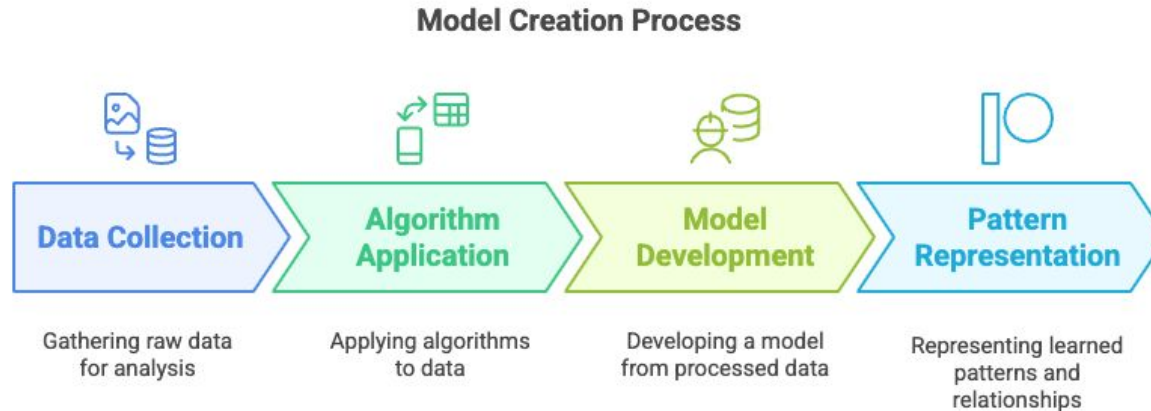
Recap: Supervised Learning

Supervised learning is a category of machine learning that uses **labeled datasets** to **train** algorithms to **predict outcomes** and **recognize patterns**.

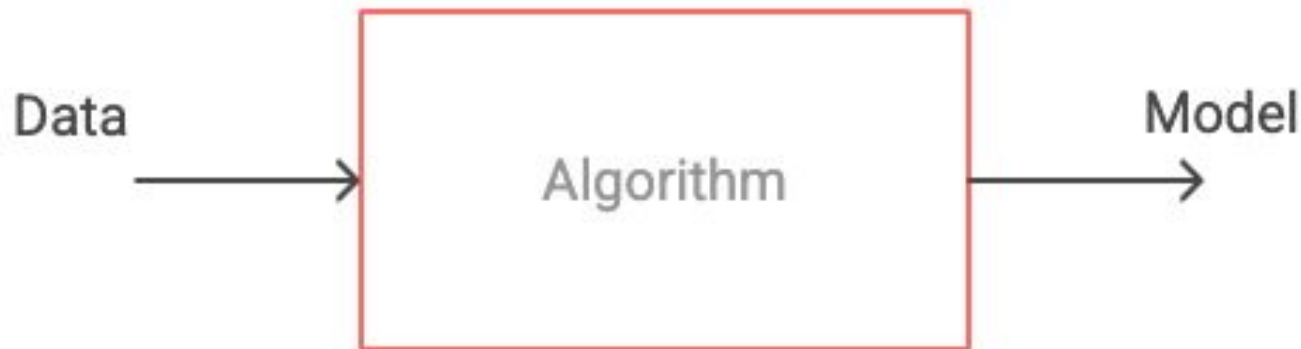


Algorithm vs Model

A **model** is the outcome of **training an algorithm on data**; it represents the learned patterns, relationships, or predictions based on the training process.

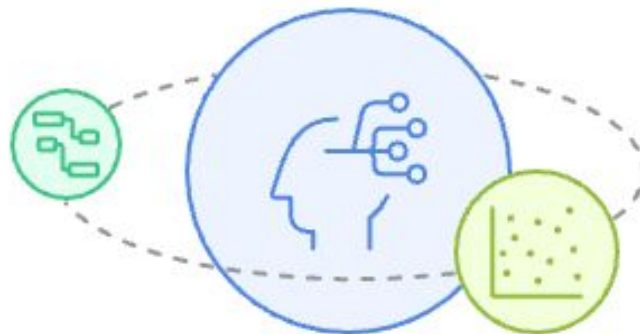


Algorithm vs Model



Types of Supervised Machine Learning (SML) Applications

Classification vs Regression



Classification

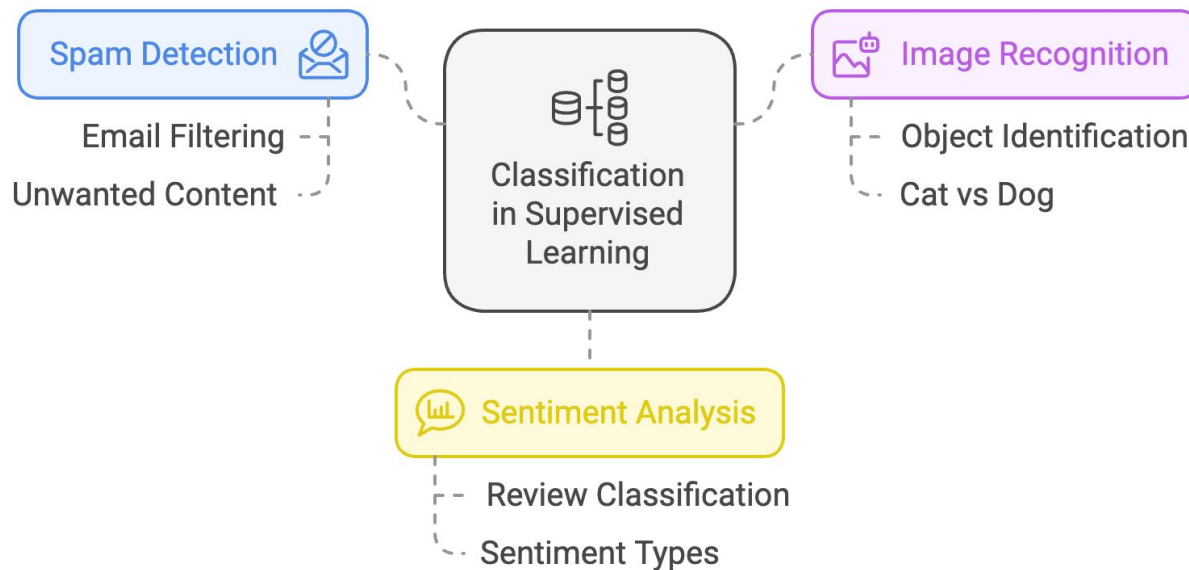
Assigns labels to
data points based
on features

Regression

Predicts
continuous values
based on input
data

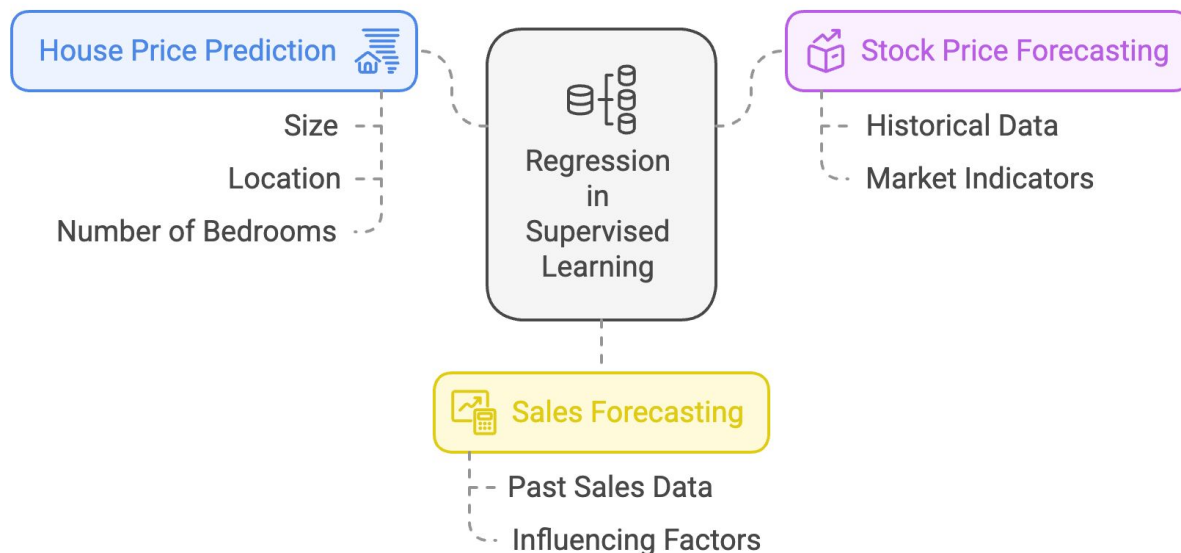
Classification

Classification is a type of supervised learning where the goal is to predict the categorical label of new observations based on past observations.



Regression

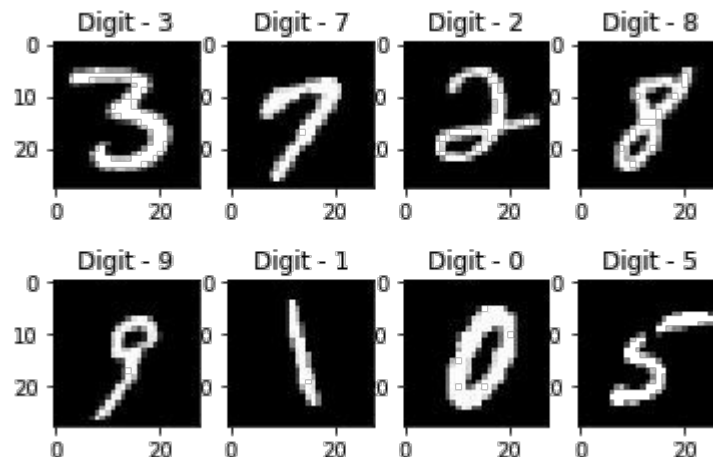
Regression is another key type of supervised learning that focuses on predicting continuous numerical values rather than categorical labels.



Examples of SML Applications

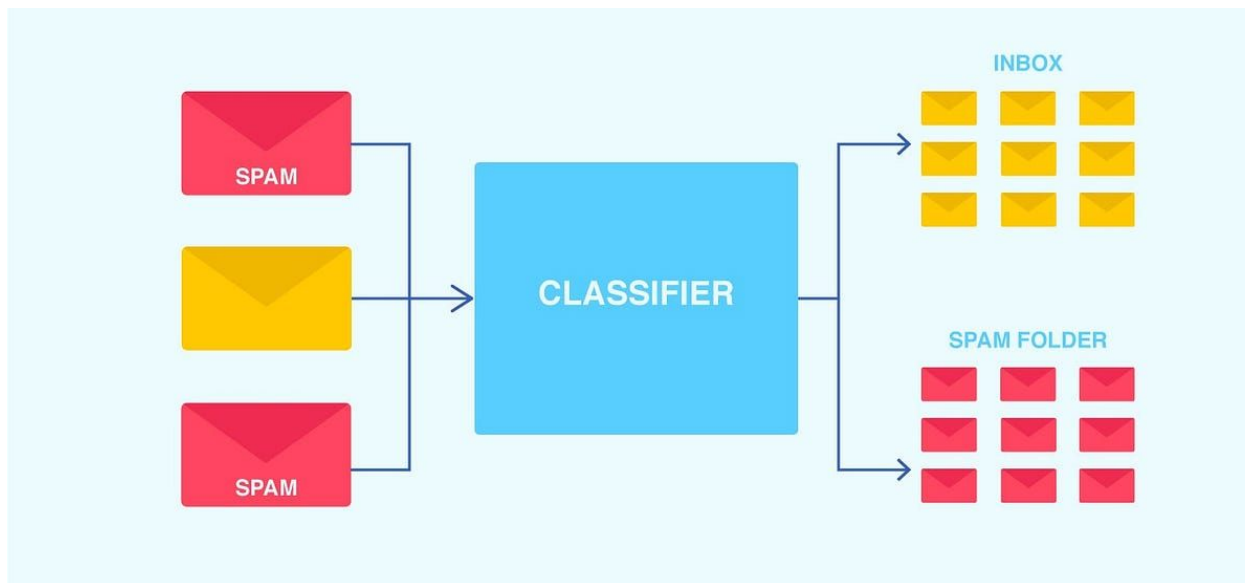
Optical Character Recognition (OCR)

The model is able to identify handwritten characters and classify each image as a character. In this case, we are classifying the number digits.



Email Prioritization

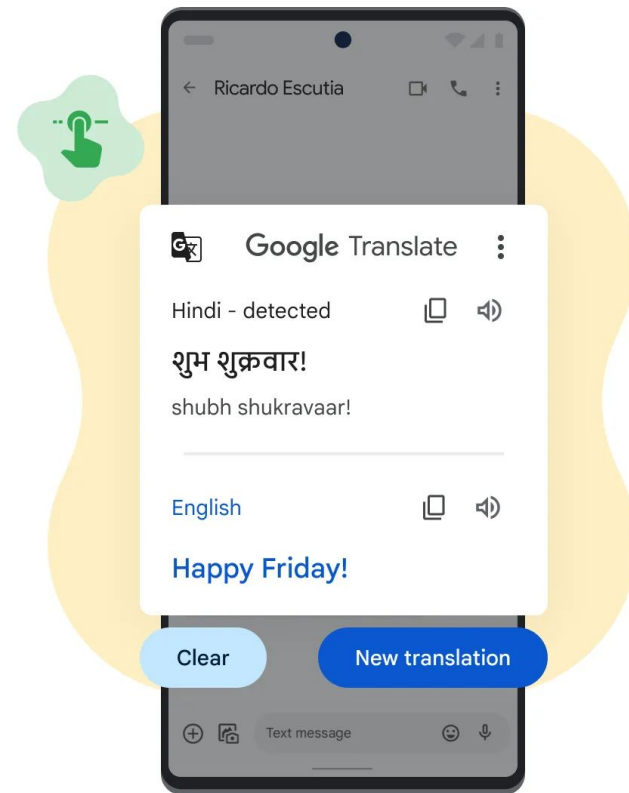
The model is able to successfully detect which of the arriving emails go to spam and which to the primary inbox.



Language Translation

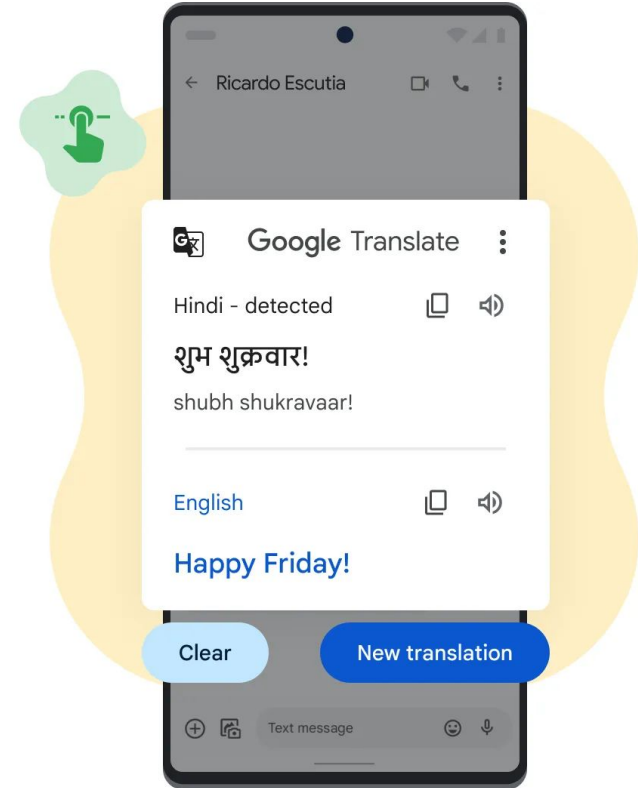
The model is able to take in a sequence in one language and output a sequence of the same information in a different language.

Would this be classification or regression?



Language Translation

Language translation is a **classification** problem in because it involves predicting the next word (or token) from a **predefined vocabulary**, which can be seen as a list of possible "**classes**." Each word in the vocabulary is effectively a "**class**" that the model selects based on the input context.

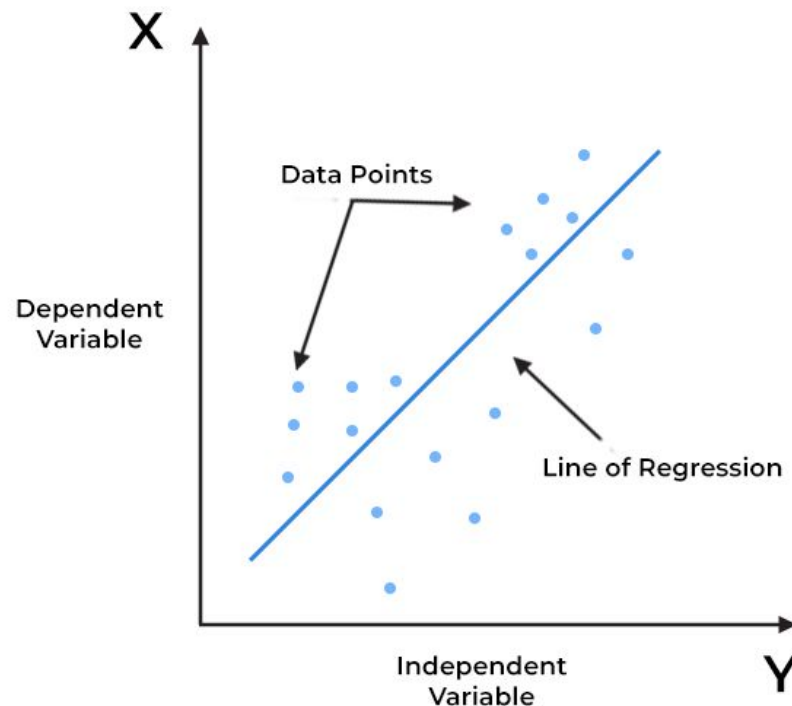


Linear Regression

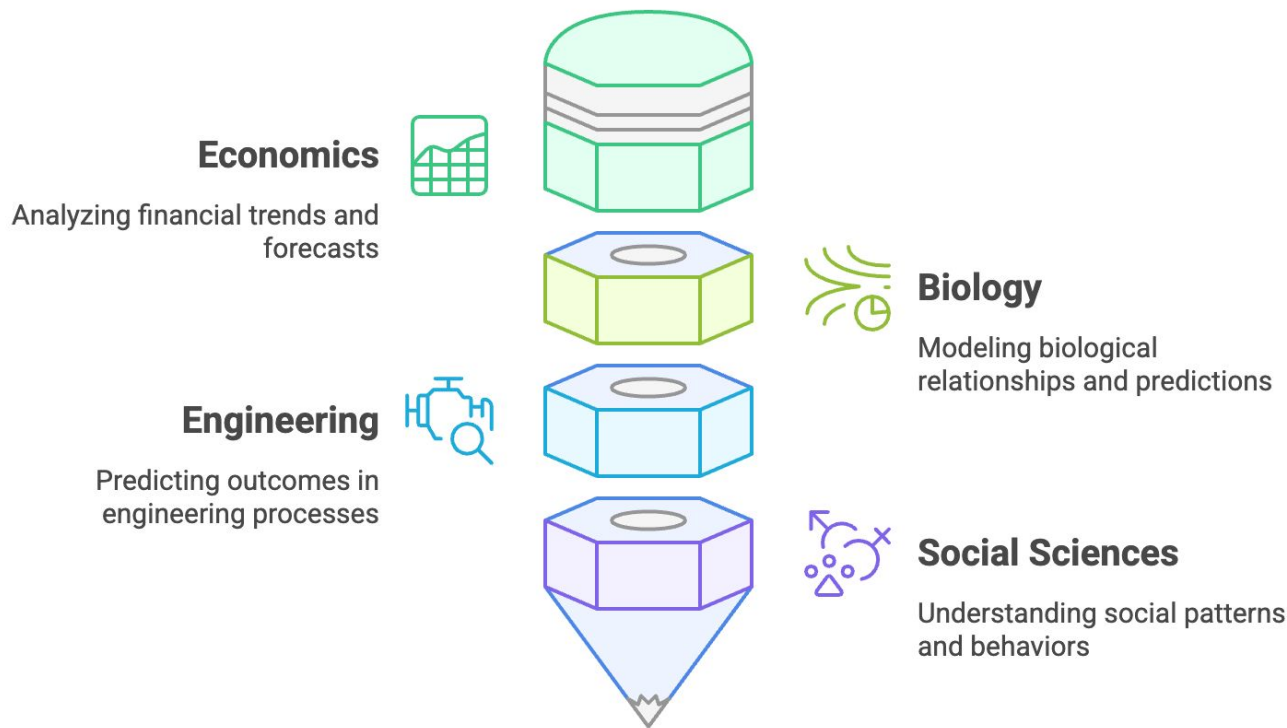
Linear Regression

Linear regression is a statistical method used to **model the relationship** between a **the target variable** and a **feature variable** by a **line**.

Based on the training data points, we try to create a line that **best models the relationships** between the input feature and the output feature.

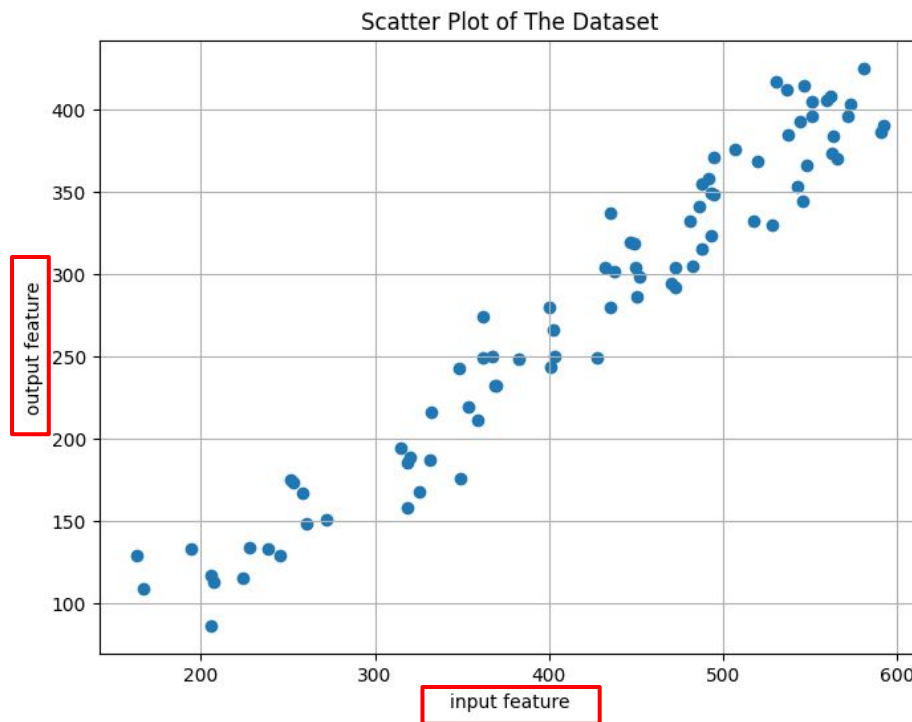


Applications of Linear Regression



Linear Regression

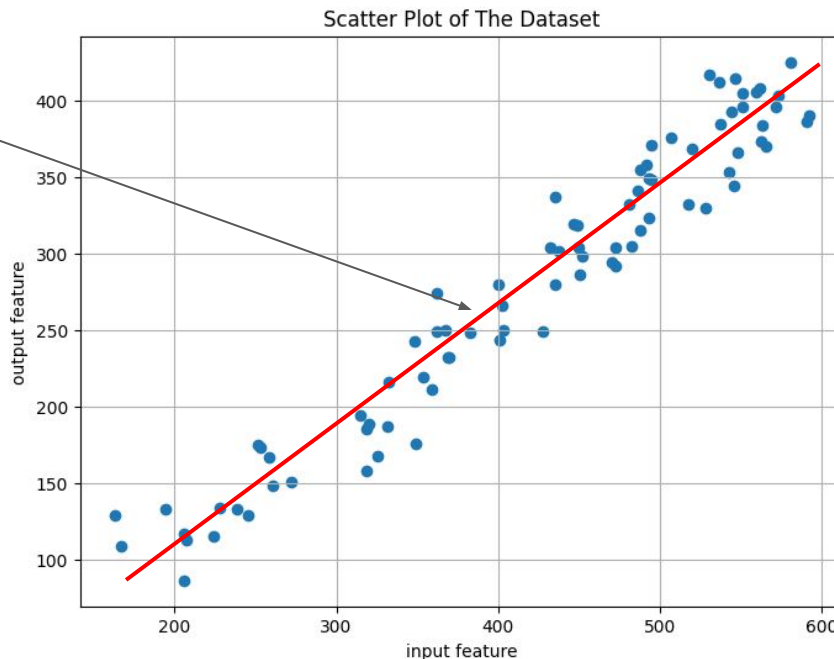
Given that our dataset has one input feature and an output feature, let's plot our data.



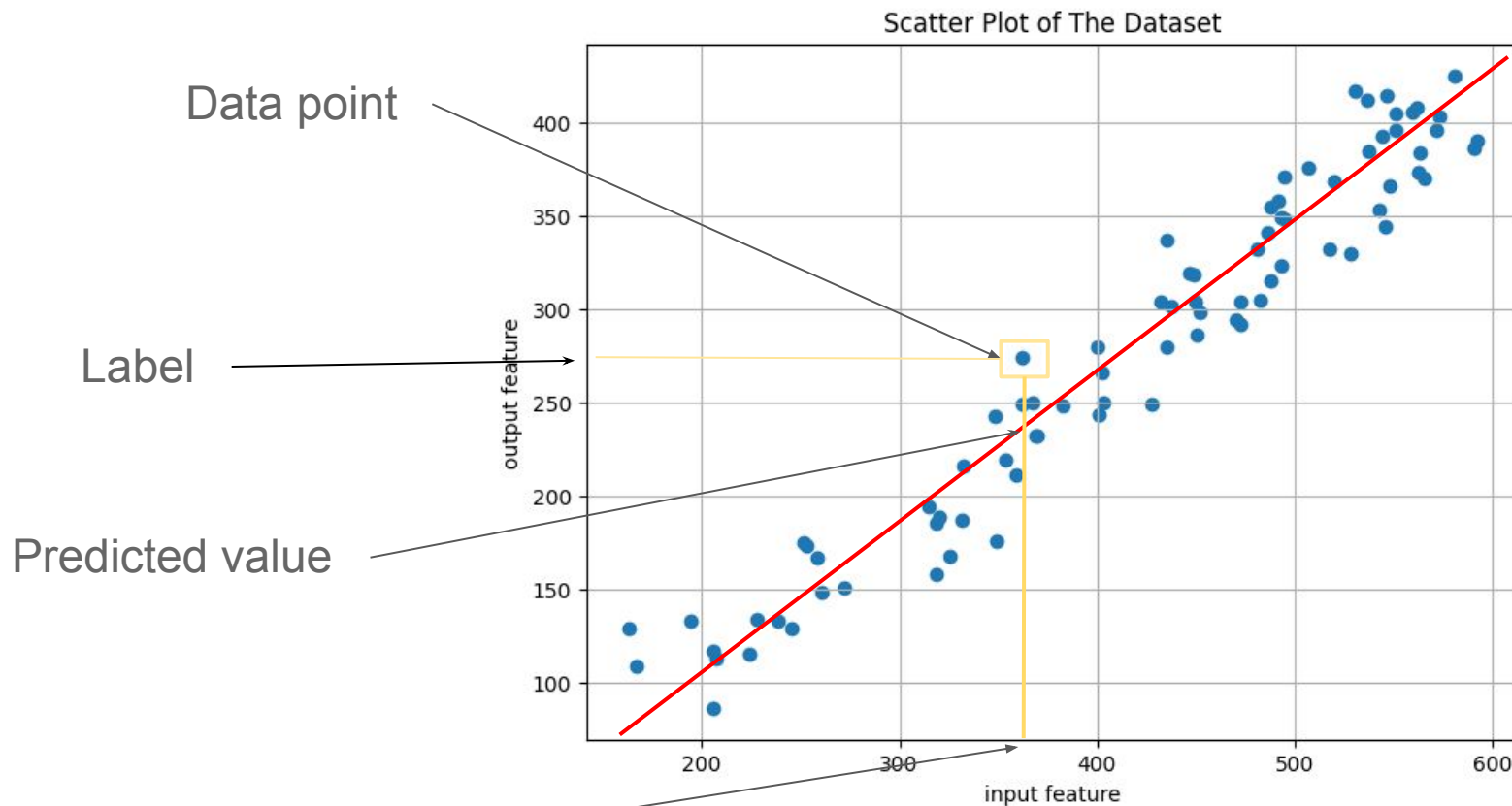
Linear Regression

We want to create a line that models the training data points in the lowest error possible. We call the modeled relationship: “the best fit line”.

- Best fit line
- Predicted line



Linear Regression

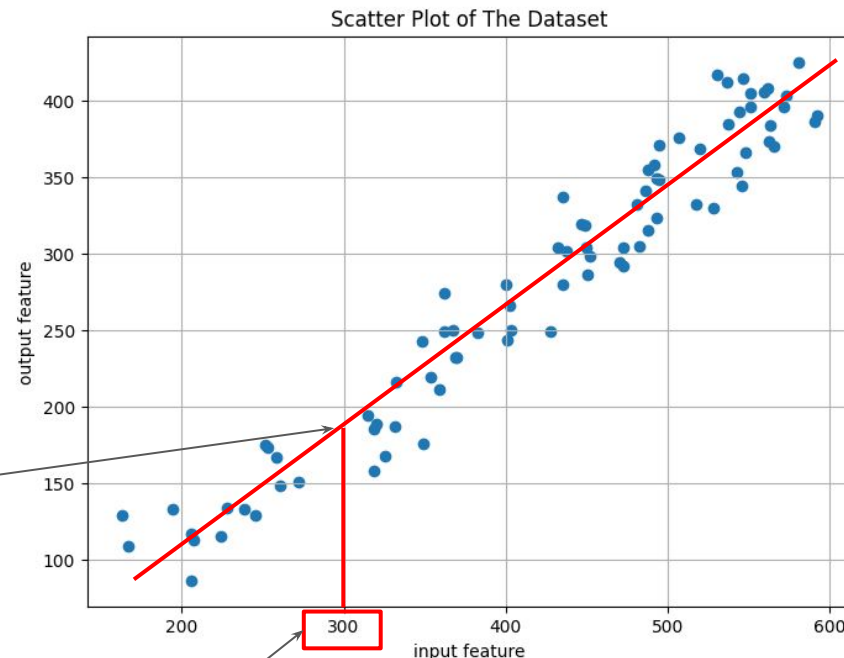


Linear Regression

We want to create a line that will not only model the current data points, but will also allow us to predict future outputs with high accuracy.

Predicted value

New input (value not seed in the dataset)

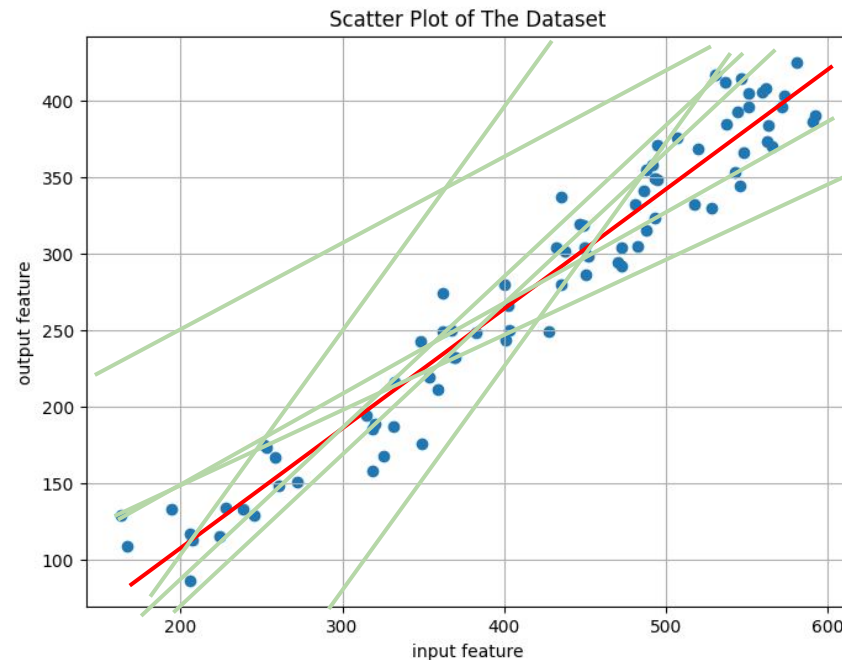


Linear regression

To get to the best fit line, we will go through multiple iterations of updates.

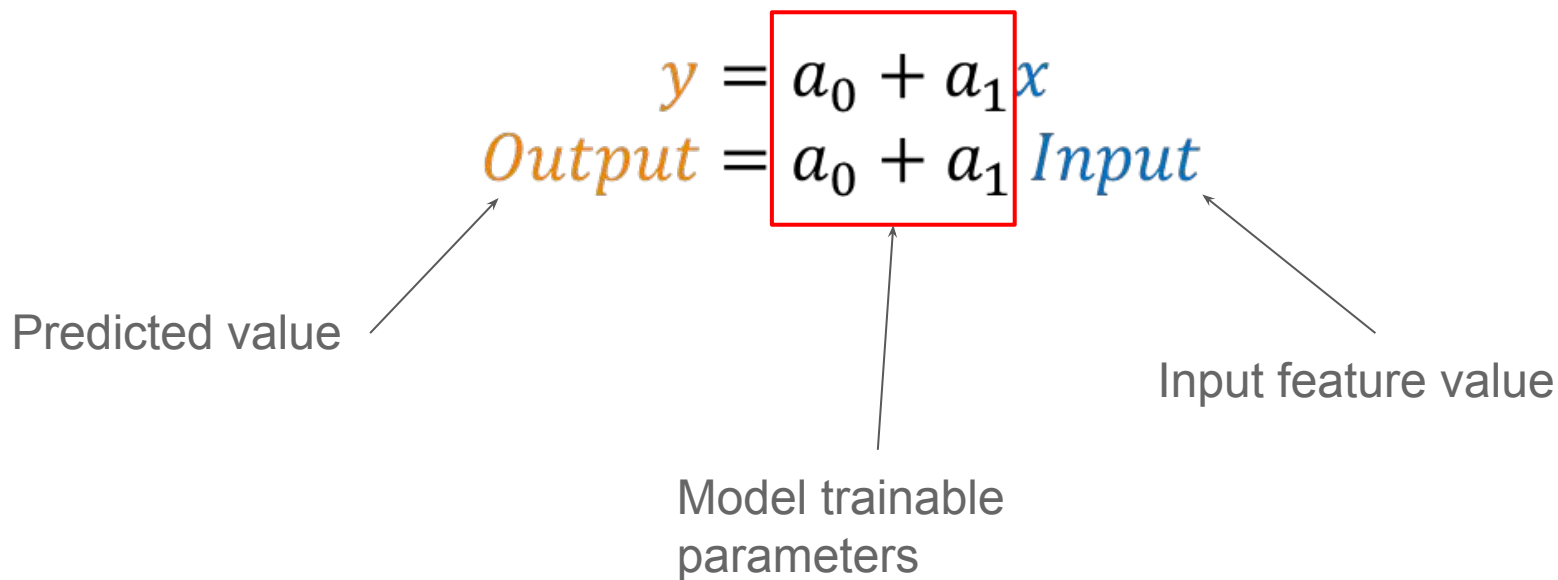
What is the model updating (or “learning”)?

In the case of creating a line, we update the **slope**, and **y-intercept**.



Linear Regression

Since the model is trying to create a best fit line, it is optimizing the equation of a line.



The diagram illustrates the linear regression equation $y = a_0 + a_1x$ and its components. The equation is shown in two forms: $y = a_0 + a_1x$ and $\text{Output} = a_0 + a_1 \text{Input}$. The term y is labeled as the Predicted value, x as the Input feature value, and the coefficients a_0 and a_1 as the Model trainable parameters. Arrows point from the labels to the corresponding parts of the equation.

$$y = a_0 + a_1x$$
$$\text{Output} = a_0 + a_1 \text{Input}$$

Predicted value

Model trainable parameters

Input feature value

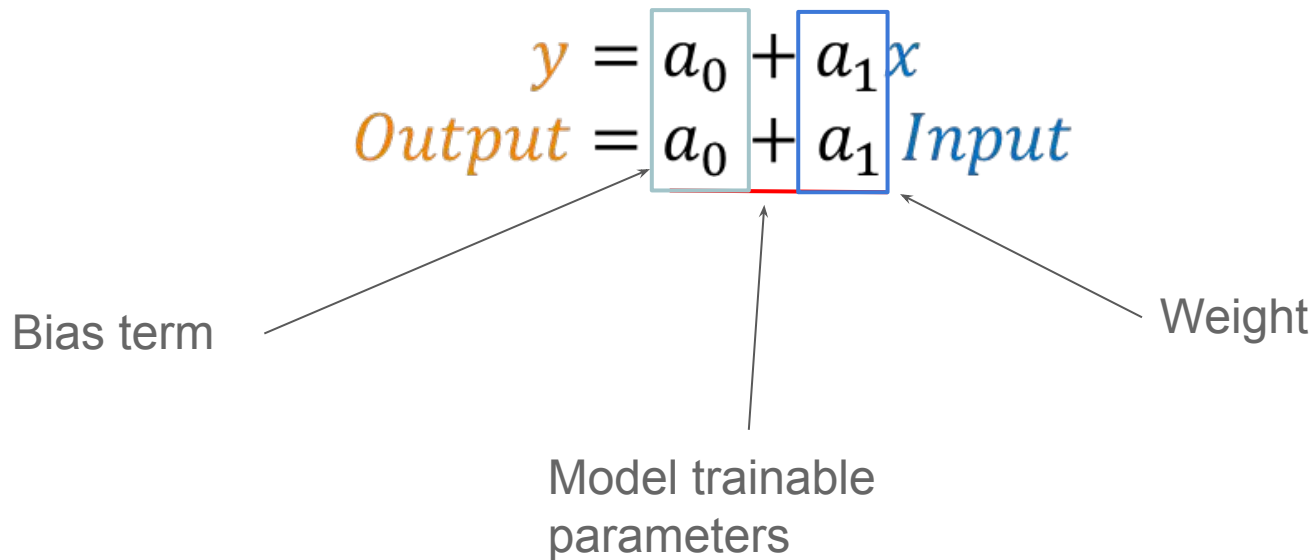
“Learning” of machine learning algorithms

We call the variables being updated during the training process as “**trainable parameters**” or “**weights**”. In some cases, we also have a “**bias**” term as a trainable parameter.

- Trainable parameters because they are getting “trained” or **updated during the learning process**.
- Weights because they also **carry the importance and contribution** of each input value.
- Bias because it helps **direct the predictions** of the model for higher accuracy.

Linear Regression

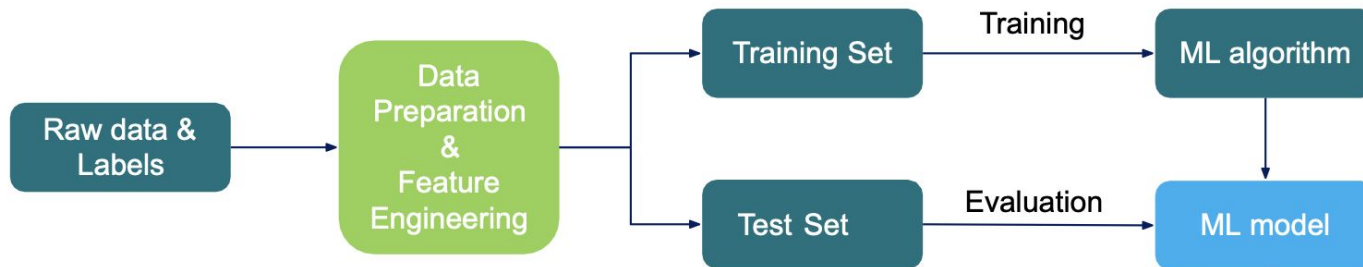
In the case of linear regression, we have both a weight and a bias in our model.



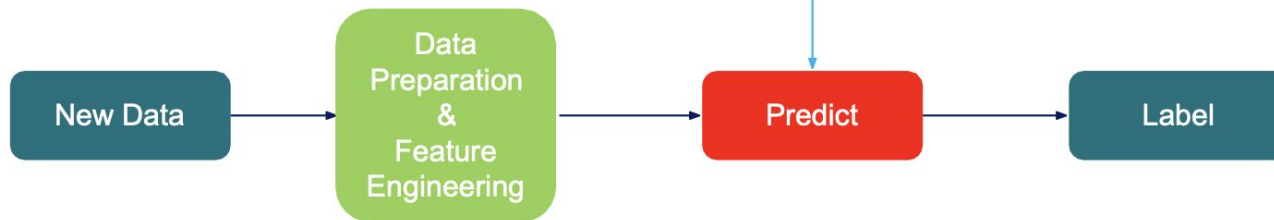
How SML algorithms learn

Learning flow of SML models

TRAINING/ LEARNING



PREDICTION/ INFERENCE



SML training process

The training (or learning) process of supervised machine learning algorithms consists of 4 parts:

1. Preparing the training dataset
2. Initializing the algorithm
3. Loop over data points in the training set:
 - a. Make a prediction (Class or Number)
 - b. Calculate the error of the prediction
4. Update the algorithm parameters

Supervised Machine Learning Training Process



Preparing the Training Dataset

Gathering and organizing data for training



Initializing the Algorithm

Setting up the algorithm's initial state



Loop Over Data Points

Iterating through each data point in the dataset



Make a Prediction

Generating an output based on the input data



Calculate Prediction Error

Assessing the difference between predicted and actual values



Update Algorithm Parameters

Adjusting the algorithm based on error feedback

Preparing the training dataset

The data preparation steps, as covered in the previous chapters, need:

1. EDA and feature tuning
2. Transformation to numerical representation
3. Split the dataset for training and testing

Data Preprocessing Steps

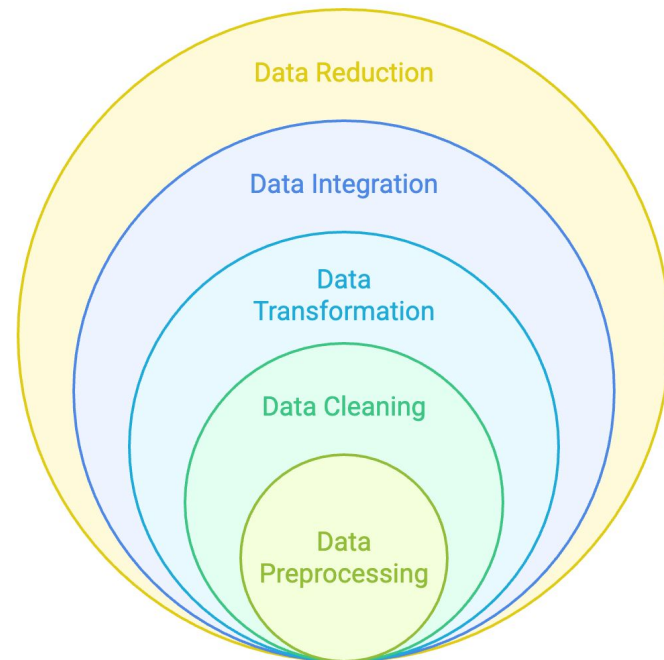
Simplifies dataset without losing information

Combines data from different sources

Normalizes and encodes data

Addresses inconsistencies and errors

Core process for data analysis



Training Linear Regression

We mentioned that linear regression has one bias term and one weight that needs to be updated using the training loop. Let's see how that happens.

First, we will use a dataset of randomly created points.

pandas_dataset

	x	y
0	167.054441	109.261875
1	163.016947	129.409473
2	205.646118	86.723659
3	194.672282	133.125094
4	224.475833	115.167052
...	...	...
78	580.569550	424.985420
79	562.728538	373.092425
80	592.653007	390.109725
81	590.391685	386.227105
82	546.401845	414.371075

83 rows x 2 columns



Training Linear Regression

Since the dataset is random and does not actually hold any information, we don't need to perform EDA.

We will only pre-process the data by normalizing it.

```
import numpy as np
x = pandas_dataset['x']
y = pandas_dataset['y']

# normalize the input feature
x_mean = np.mean(x)
x_std = np.std(x)
x_normalized = (x - x_mean) / x_std
```

pandas_dataset

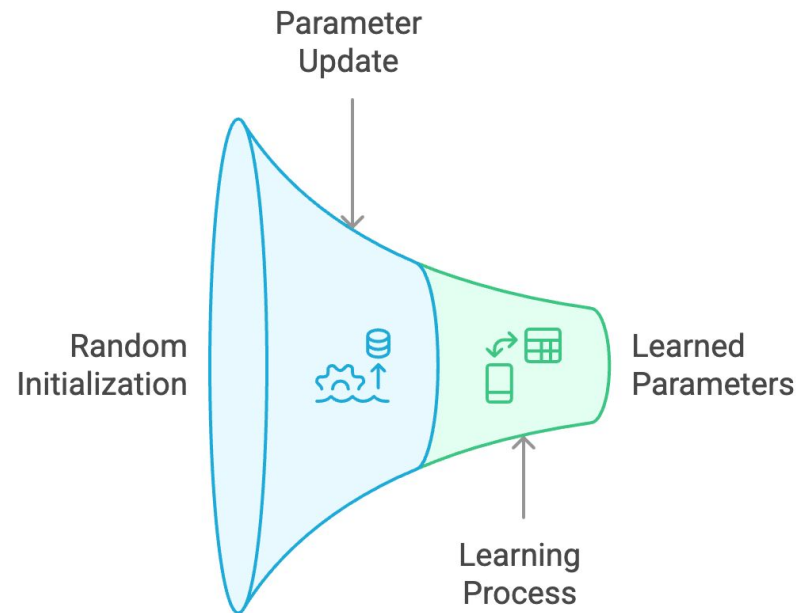
	x	y
0	167.054441	109.261875
1	163.016947	129.409473
2	205.646118	86.723659
3	194.672282	133.125094
4	224.475833	115.167052
...	...	...
78	580.569550	424.985420
79	562.728538	373.092425
80	592.653007	390.109725
81	590.391685	386.227105
82	546.401845	414.371075

83 rows x 2 columns

Initializing the algorithm

Every machine learning algorithm holds its learned patterns and connections within its **parameters**, often referred to as “**trainable parameters**”.

These **trainable parameters**, at the beginning of the training process, are **randomly initialized** to be updated and “**learned**” during the training process.



Linear Regression Example

So, we will choose random initializations for our trainable parameters.

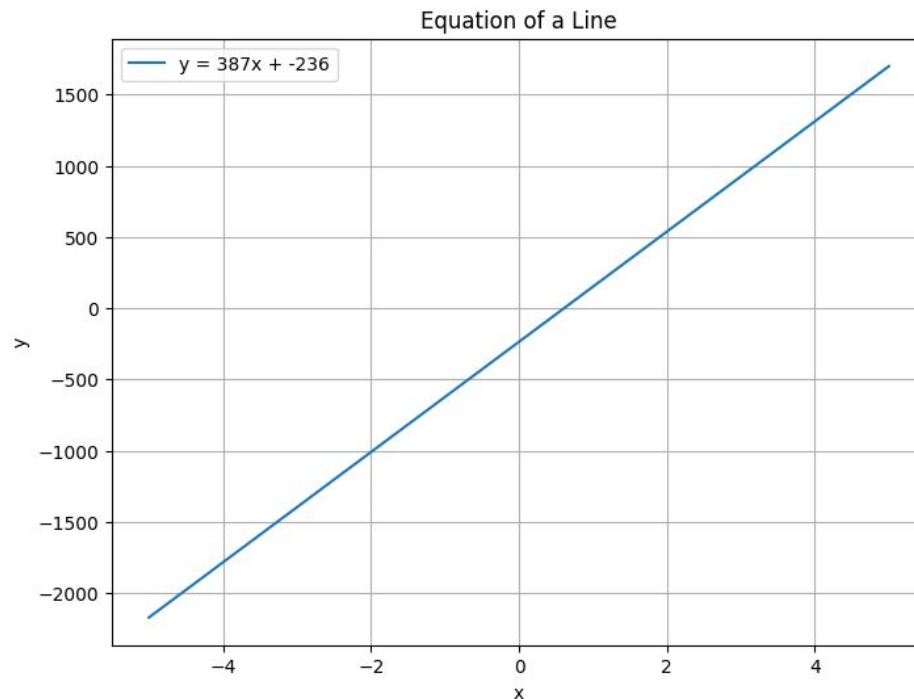
```
import random
initial_a0 = random.randint(-500, 500)
initial_a1 = random.randint(-500, 500)
print(f"initial_a0: {initial_a0}, initial_a1: {initial_a1}")

initial_a0: 188, initial_a1: 42
```



Linear Regression Example

Let's visualize initialized line



Looping over data points

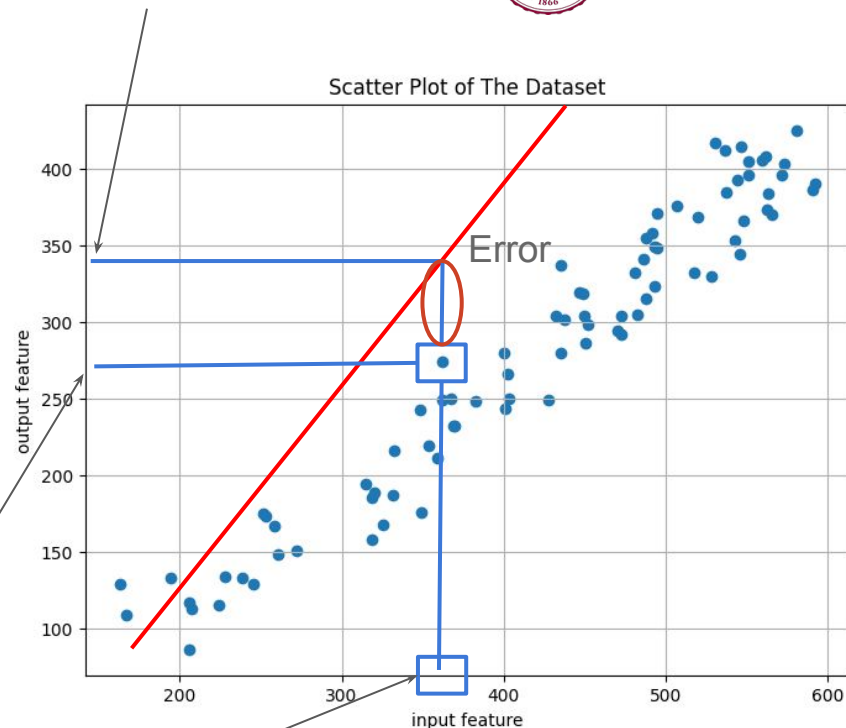
For every data point, we feed the data points to the model so it can make a prediction.

This prediction would not be accurate, so it contains some **error**.

Predicted value

Real value

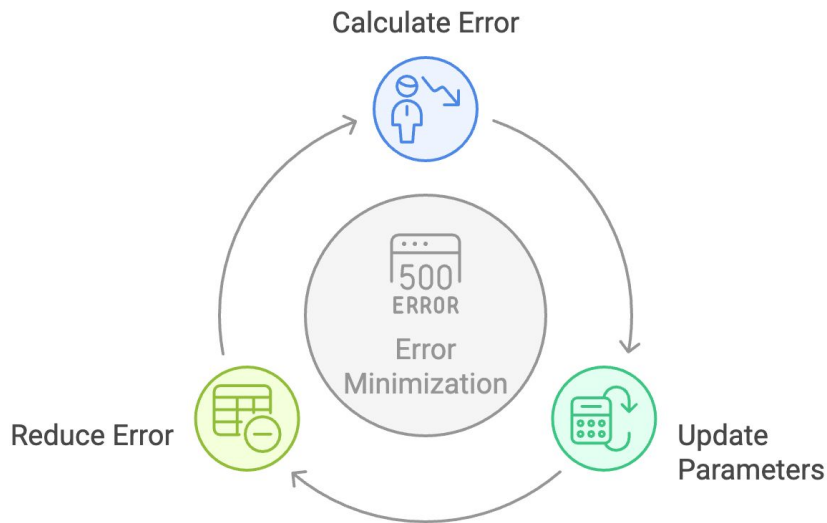
Input data point



Looping over data points

This error that we got, which is the difference between real and predicted values, should guide our update of the trainable parameters.

The goal is to update the trainable parameters so that their update results in a lower error.

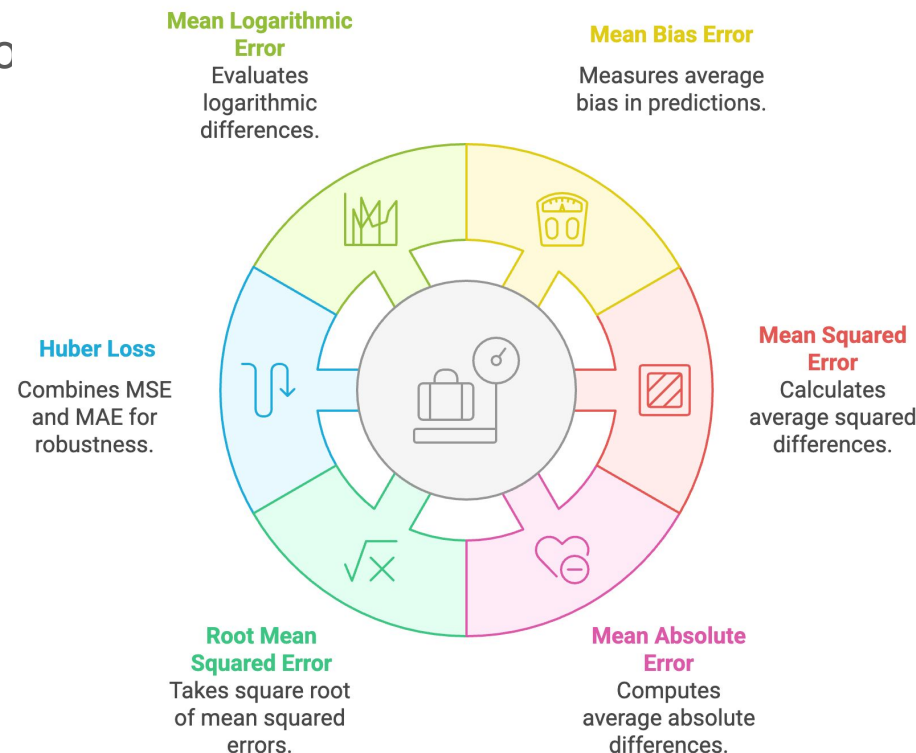


Error calculation

For regression tasks, we have multiple options to compare the real and predicted values:

1. Mean Bias Error (MBE)
2. Mean Squared Error (MSE)
3. Mean Absolute Error (MAE)
4. Root Mean Squared Error (RMSE)
5. Huber Loss
6. Mean Logarithmic Error (MLE)
7. ...

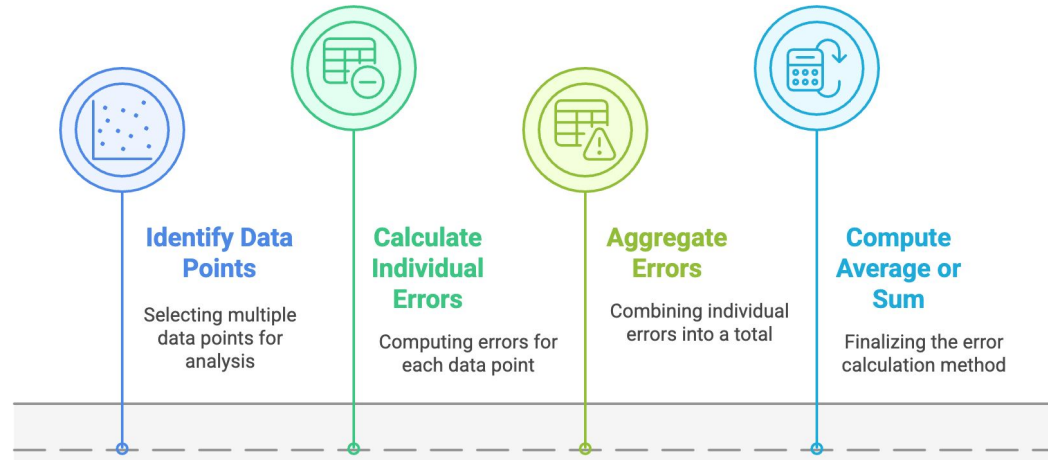
Regression Error Metrics



Error calculation - Loss function

If we calculate the error for more than one data point at a time, this is done using a **loss function**.

The loss function **aggregates the individual errors** across the selected data points, typically by computing **the average or sum of the errors** using a specified error function (e.g., MSE, MAE).



Error Calculation - Mean Bias Error

Measures the average difference between predicted and actual values. It indicates the direction of the error (positive or negative bias).

- Positive MBE: Overestimation.
- Negative MBE: Underestimation.

Useful for understanding bias in predictions but not ideal as a standalone metric because it doesn't capture the magnitude of errors.

Error Function: $\text{MBE}_i = y_i - \hat{y}_i$

Loss Function: $\text{MBE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$

Linear Regression Example - MBE

In the linear regression model we initialized earlier, let's try to get a prediction from the first data point in the training set and calculate its MBE.

```
first_data_point = x_normalized[0]
label = pandas_dataset['y'][0]

print(f"first_data_point: {first_data_point}, label: {label}")
```

```
first_data_point: -1.771487256006177, label: 36.46414833126471
```

```
prediction = initial_a1 * first_data_point + initial_a0
print(f"prediction: {prediction}")
```

```
prediction: 113.59753524774057
```

```
mean_bias_error = label - prediction
print(f"mean_bias_error: {mean_bias_error}")
```

```
mean_bias_error: -77.13338691647586
```

Error Calculation - Mean Squared Error

Measures the average squared differences between predicted and actual values. Squaring emphasizes larger errors, making it **sensitive to outliers**.

Use this error when you want to penalize large errors more heavily or when outliers are meaningful.

Error Function:
$$\text{MSE}_i = (y_i - \hat{y}_i)^2$$

Loss Function:
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Linear Regression Example - MSE

Using the MSE, we see a significant difference in the error.

```
mean_squared_error = (label - prediction) ** 2  
print(f"mean_squared_error: {mean_squared_error}")
```

```
mean_squared_error: 5949.559377206769
```

Both MSE and MBE can guide the update of trainable parameters during model training, but they serve different purposes. However **MSE is more commonly used to minimize overall prediction error**, and MBE is used to **provide insight** into whether the model consistently overestimates or underestimates.

Error Calculation - Mean Absolute Error

Measures the average of **absolute differences between predicted and actual values**. It treats all errors equally, making it robust to outliers.

Useful for when you want a simple, interpretable metric that is less sensitive to outliers than MSE.

Error Function: $\text{MAE}_i = |y_i - \hat{y}_i|$ Loss Function: $\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$

Linear Regression Example - MAE

The MAE provides a more robust measure of error by treating all deviations equally, without disproportionately penalizing large errors, unlike MSE. This characteristic makes MAE less sensitive to outliers and provides a more balanced reflection of typical errors in the model.

```
mean_absolute_error = abs(label - prediction)
print(f"mean_absolute_error: {mean_absolute_error}")
```

```
mean_absolute_error: 77.13338691647586
```

Error Calculation - Root Mean Squared Error

The square root of MSE. It provides the error in the same units as the target variable, making it more interpretable than MSE.

Useful for when you want a metric in the same scale as the target variable, while still penalizing large errors more heavily.

Suppose you're predicting house prices in dollars. RMSE provides an error value (e.g., \$5,000) that is also in dollars. This tells you that, on average, your model's prediction is about \$5,000 off from the actual value.

Error Function: $\text{RMSE}_i = \sqrt{(y_i - \hat{y}_i)^2}$ Loss Function: $\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$

Linear Regression Example - RMSE

In this case, we got RMSE and MAE as the same value. This makes sense because we calculating the errors for one data point.

```
root_mean_squared_error = np.sqrt(mean_squared_error)
print(f"root_mean_squared_error: {root_mean_squared_error}")
```

```
root_mean_squared_error: 77.13338691647586
```

Both are expressed in the same units as the target variable, but RMSE can sometimes overemphasize large errors, which might distort the perception of the model's performance.

Error Calculation - Huber Loss

Combines the properties of MSE and MAE. It behaves like MSE for small errors and switches to MAE for large errors, making it robust to outliers while maintaining sensitivity to small errors.

Error Function:

$$L_i(y_i, \hat{y}_i) = \begin{cases} \frac{1}{2}(y_i - \hat{y}_i)^2, & \text{if } |y_i - \hat{y}_i| \leq \delta, \\ \delta|y_i - \hat{y}_i| - \frac{1}{2}\delta^2, & \text{otherwise.} \end{cases}$$

Loss Function:

$$\text{Huber Loss} = \frac{1}{n} \sum_{i=1}^n L_i(y_i, \hat{y}_i)$$

You set the threshold (δ) according to the size of your dataset

Linear Regression Example - Huber Loss

In our case, since the error $> \delta$, then we applied the modified version of MAE as the result of the huber loss.

```
delta = 1.0
error = np.abs(label - prediction)
print(f"Error: {error}")
quadratic_loss = 0.5 * error**2
linear_loss = delta * (error - 0.5 * delta)
huber_loss_value = np.where(error <= delta, quadratic_loss, linear_loss)
print(f"huber_loss: {huber_loss_value}")
```

```
Error: 77.13338691647586
huber_loss: 76.63338691647586
```

Error Calculation - MLE

Measures the error logarithmically, which reduces the impact of large errors. This metric is useful when the target values vary over several orders of magnitude.

Useful for when handling data with widely varying scales or when large errors are undesirable but should not dominate the metric.

Error Function:
$$\text{MLE}_i = \log(1 + |y_i - \hat{y}_i|)$$

Loss Function:
$$\text{MLE} = \frac{1}{n} \sum_{i=1}^n \log(1 + |y_i - \hat{y}_i|)$$

Linear Regression Example - MLE

As you can see MLE tends to be much smaller than other error metrics like MSE, MAE, or RMSE. This is because it focuses on **relative error rather than outlier impact**.

```
mean_logarithmic_error = np.mean(np.log(np.abs(label - prediction)))  
print(f"mean_logarithmic_error: {mean_logarithmic_error}")
```

```
mean_logarithmic_error: 4.345536220773281
```

Regression Error Functions Recap

Error Function	When to Use
MBE	To evaluate bias in predictions.
MSE	To penalize large errors (sensitive to outliers).
MAE	For robustness to outliers and interpretable metrics.
RMSE	To penalize large errors while keeping units interpretable.
Huber Loss	For balanced sensitivity to small errors and robustness to outliers.
MLE	For data with a wide dynamic range or when large errors need to be down-weighted.

Updating Algorithm Trainable Parameters

Now, the algorithm is initialized, and we made a prediction (or a set of predictions) and calculated their error using the chosen loss function.

We want to utilize this error to perform an update in to the trainable parameters.

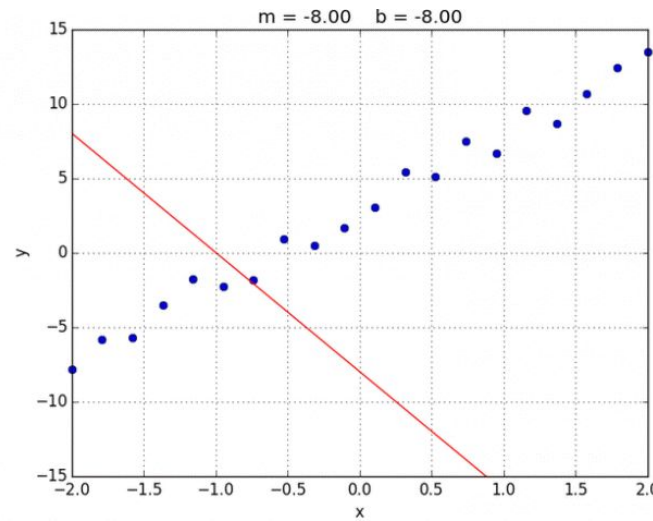
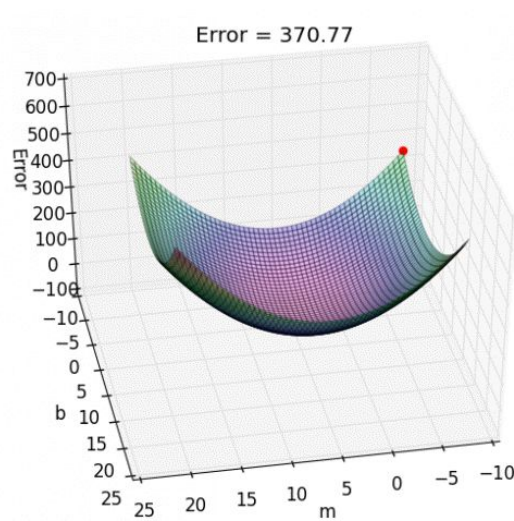
Intuitively, we want to update the trainable parameters in a way that will **minimize the error**.

This process is performed using a family of algorithms called **optimization functions**.

Optimization Algorithms - Gradient Descent

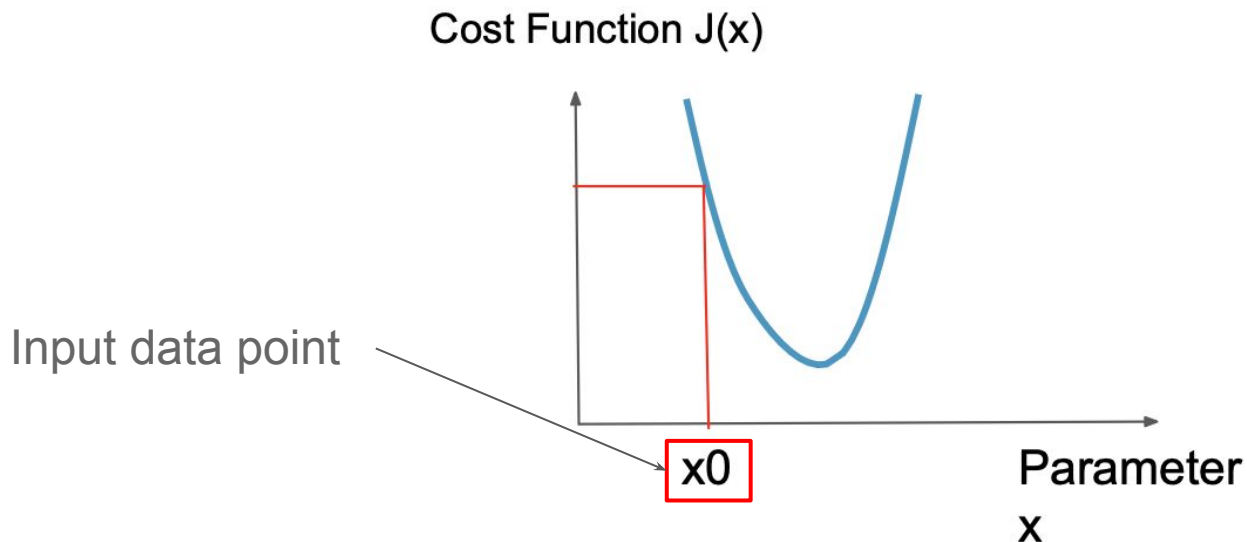
For now, we will focus on the most popular optimization algorithm: **Gradient Descent**, which can be used in the training of **ANY** machine learning algorithm.

Gradient descent is simply used to find the values of **a function's parameters** (coefficients) that **minimize a cost function as much as possible**.



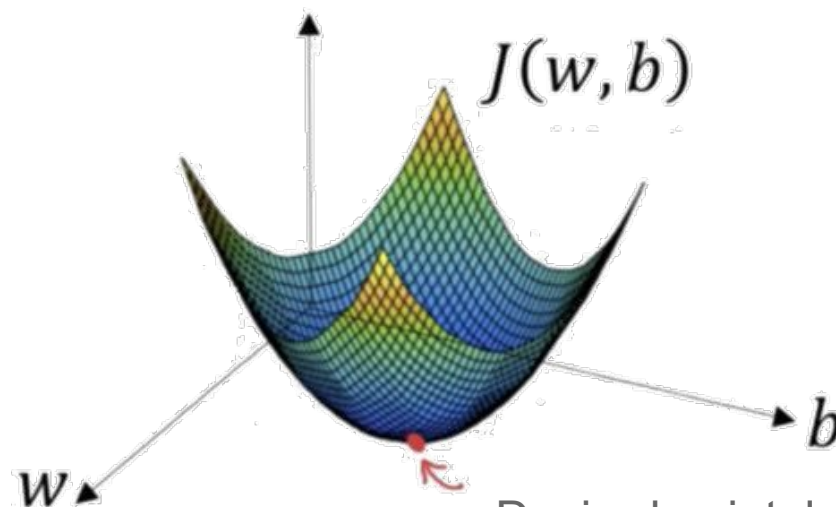
Optimization Algorithms - Gradient Descent

If we were to plot the **cost (or loss) function with respect to the change in value of one trainable parameter**, we would get a 2D curve.



Optimization Algorithms - Gradient Descent

If we were to plot the **error with respect to the change in value of two trainable parameter**, we would get a 3D curve.

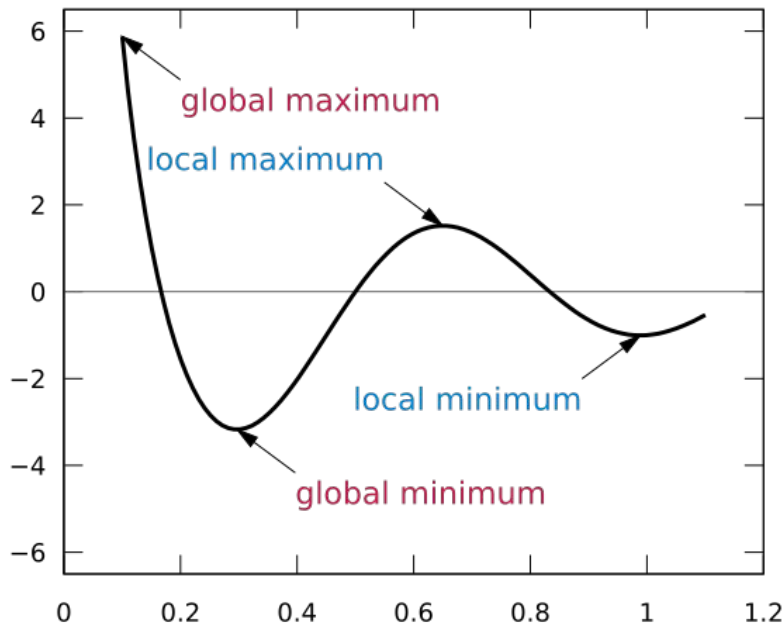


Desired point: lowest value of the error

Optimization Algorithms - Gradient Descent

To get the lowest error, we should get the minimum of the loss (or cost) function.

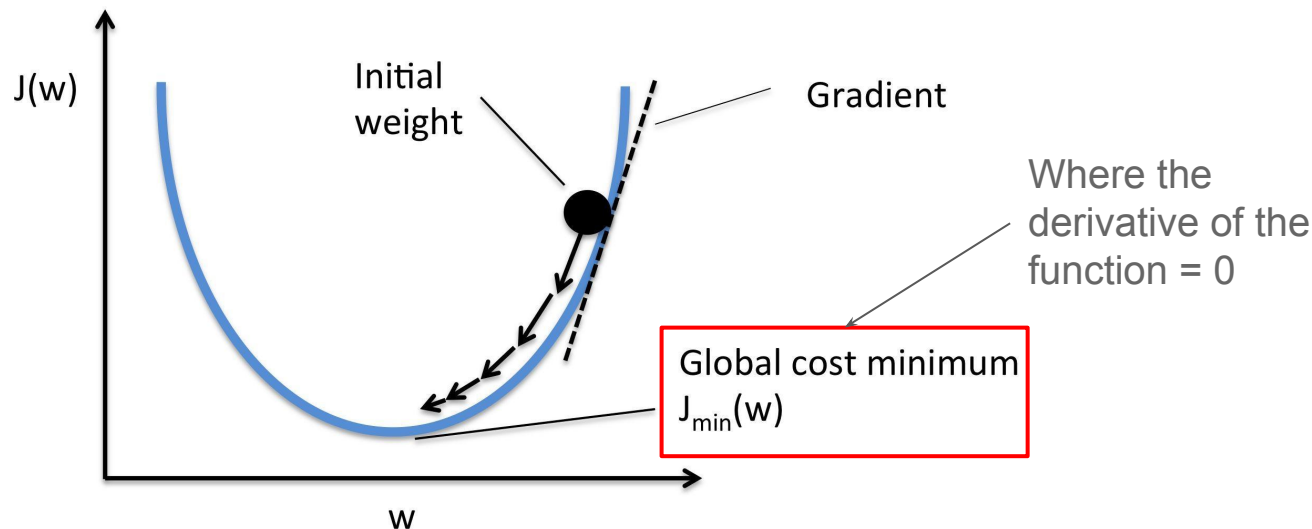
How do we get the global minimum of a function?



Optimization Algorithms - Gradient Descent

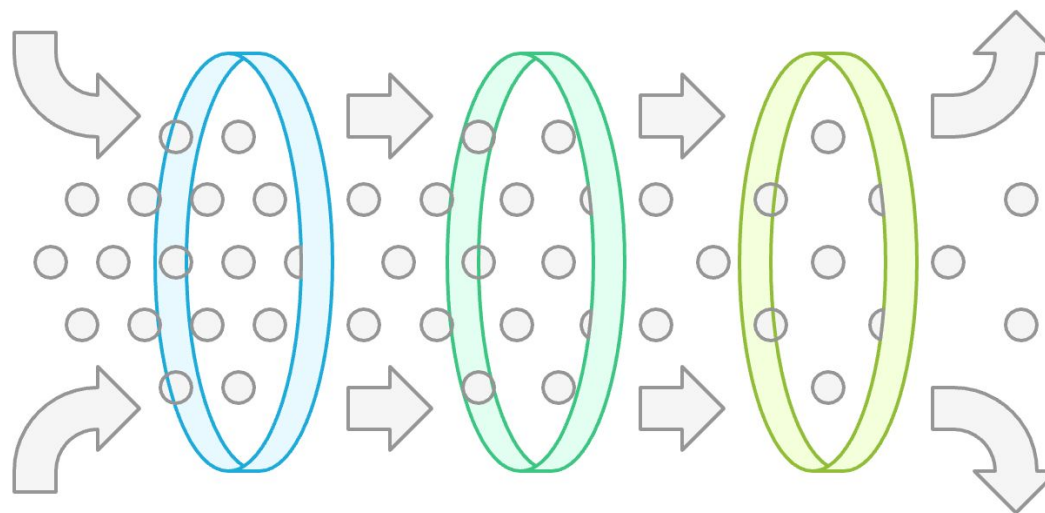
The global minimum of a function is where the derivative of the function is $=0$.

Thus, we want to move the parameter closer to where the the loss is at its minimum through incremental steps which consider the derivative of the loss function.



Optimization Algorithms - Gradient Descent

Thus, the update moves as follows:



**Calculate
Derivative**

Assess slope of loss
function

**Adjust
Parameters**

Move closer to
minimum

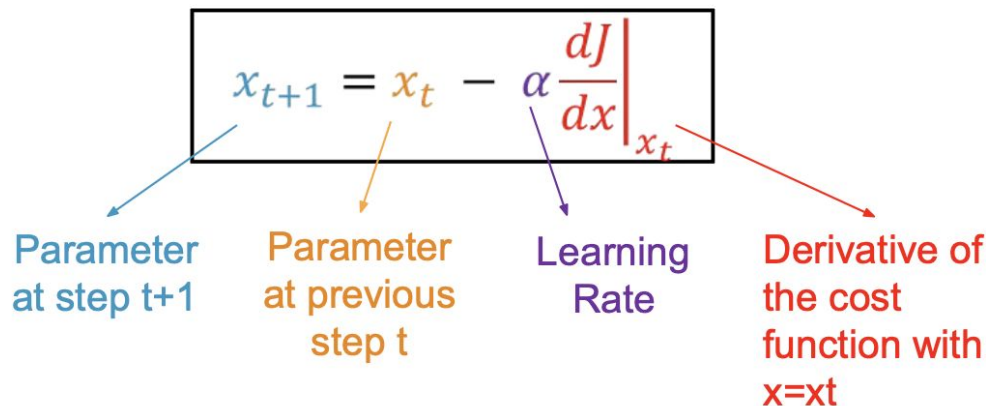
Iterate Process

Repeat until
convergence

Optimization Algorithms - Gradient Descent

Gradient Descent (GD) is an iterative process where you start at a **coefficient's initial point (x_0)** and you move step by step until you reach the minimum of the loss function.

The update rule of your position is given by this formula:

$$x_{t+1} = x_t - \alpha \left. \frac{dJ}{dx} \right|_{x_t}$$


The diagram illustrates the gradient descent update rule. A central box contains the formula $x_{t+1} = x_t - \alpha \left. \frac{dJ}{dx} \right|_{x_t}$. Four arrows point from labels below to parts of the formula: a blue arrow from 'Parameter at step t+1' to x_{t+1} , an orange arrow from 'Parameter at previous step t' to x_t , a purple arrow from 'Learning Rate' to α , and a red arrow from 'Derivative of the cost function with $x=x_t$ ' to the derivative term $\left. \frac{dJ}{dx} \right|_{x_t}$.

Parameter at step t+1

Parameter at previous step t

Learning Rate

Derivative of the cost function with $x=x_t$

Optimization Algorithms - Gradient Descent

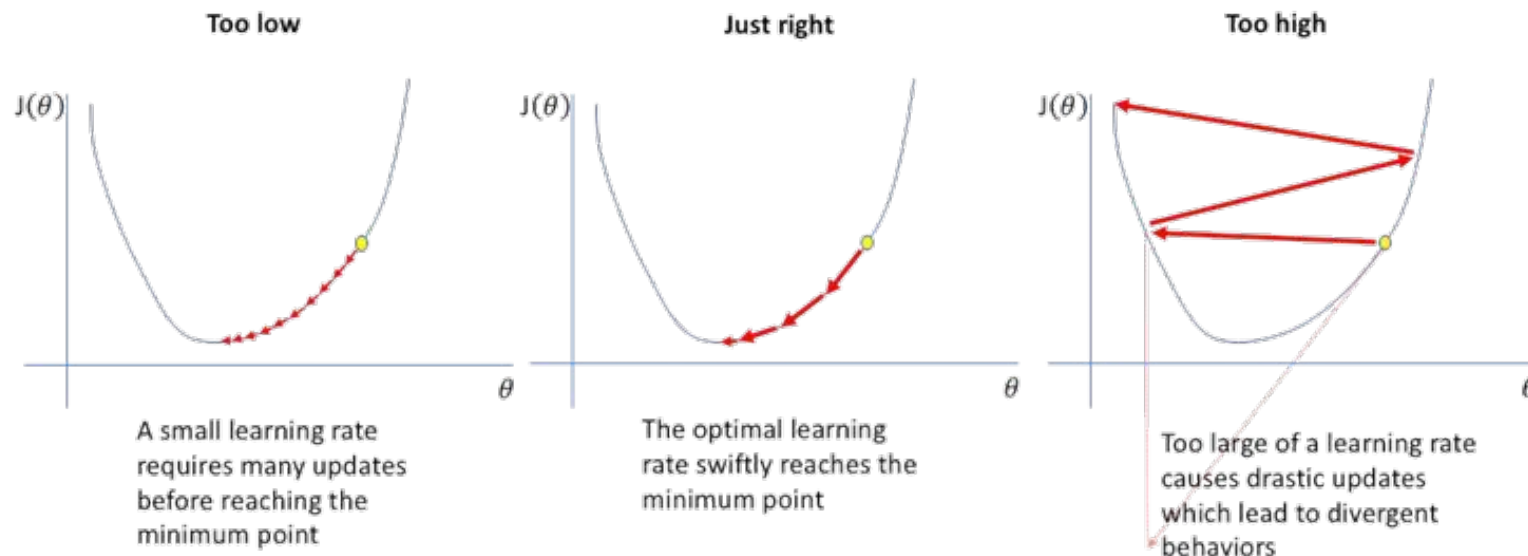
Let's dissect the update rule of gradient descent:

$$\text{New_value} = \text{old_value} - \text{learning_rate} * \text{derivative_of_cost_function_wrt_param}$$

1. *New_value*: The updated value of the parameter.
2. *old_value*: The previous value of the parameter. In the case of the first update, this is the random initialization value of the parameter.
3. *learning_rate*: The rate of update (how fast or slow) we will move towards the optimal coefficients/parameters.
4. *derivative_of_cost_function_wrt_param*: The derivative of the cost function used to direct the update closer to the global minimum.

Optimization Algorithms - Gradient Descent

The choice of the learning rate affects the size of the steps we are taking to get to the minimum.





Optimization Algorithms - Gradient Descent

Let's take MSE as the choice for our loss function and compute its derivative with respect to the trainable parameters

$$Cost = \frac{1}{n} \sum_{i=1}^n (y - y_{pred})^2 = \frac{1}{n} \sum_{i=1}^n (y - (a_0 + a_1 x))^2$$

$$\frac{\partial Cost}{\partial a_0} = -\frac{2}{n} \sum_{i=1}^n (y - (a_0 + a_1 x)) \rightarrow \frac{\partial Cost}{\partial a_0} = -\frac{2}{n} \sum_{i=1}^n (y - y_{pred})$$

$$\frac{\partial Cost}{\partial a_1} = -\frac{2}{n} \sum_{i=1}^n x(y - (a_0 + a_1 x)) \rightarrow \frac{\partial Cost}{\partial a_1} = -\frac{2}{n} \sum_{i=1}^n x(y - y_{pred})$$



Updating Algorithm Trainable Parameters

Parameter Update Strategies

Depending on the size and complexity of c parameters:

- Every data point
- Every batch of data points
- After the ingestion of all the training d

Identify Data Size and Complexity

Assess the scale and intricacy of data

Choose Update Frequency

Decide on the timing of parameter updates

Update After Every Data Point

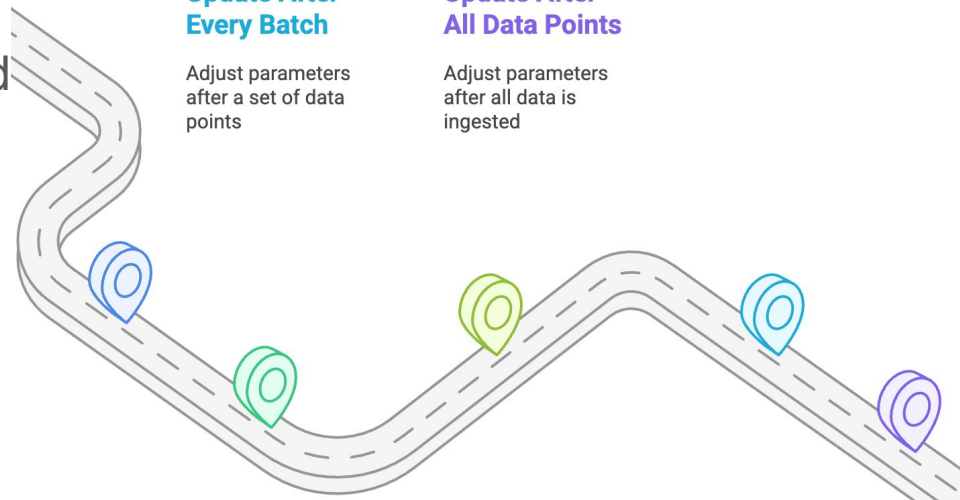
Adjust parameters after each data point

Update After Every Batch

Adjust parameters after a set of data points

Update After All Data Points

Adjust parameters after all data is ingested

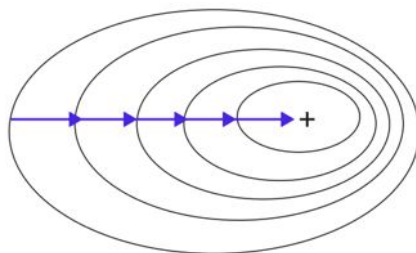


When do we perform this update?

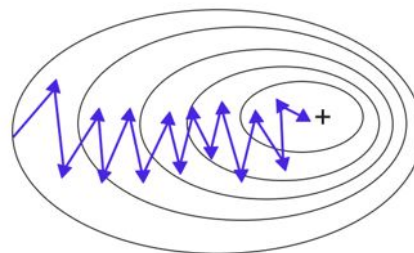
We can apply gradient descent:

1. After every data point: using **Stochastic Gradient Descent**
2. After every batch of data points: using **Batch Gradient Descent**
3. After all the training data points: using **Gradient Descent**

Batch Gradient Descent



Stochastic Gradient Descent





Linear Regression Example

Going back to our linear regression example, we can now train our algorithm by performing the parameter update loop using gradient descent.

```
def gradient_descent(x, y, learning_rate, nb_iterations, initial_a0, initial_a1):  
    n = x.shape[0] # Number of points  
    a0 = initial_a0  
    a1 = initial_a1  
  
    # Normalize x  
    x_mean = np.mean(x)  
    x_std = np.std(x)  
    x_normalized = (x - x_mean) / x_std  
  
    for i in range(nb_iterations):  
        y_pred = a1 * x_normalized + a0  
  
        a0_der = -2 * np.sum(y - y_pred) / n  
        a1_der = -2 * np.sum(x_normalized * (y - y_pred)) / n  
  
        a0 = a0 - learning_rate * a0_der  
        a1 = a1 - learning_rate * a1_der  
  
    return a0, a1
```

```
learning_rate = 0.001  
nb_iterations = 100
```

```
a0, a1 = gradient_descent(x,y,learning_rate, nb_iterations, initial_a0, initial_a1)  
print ("Equation is: y = ", a0,"+",a1,"x")
```

Equation is: y = 188.0 + 56.59849869019957 x

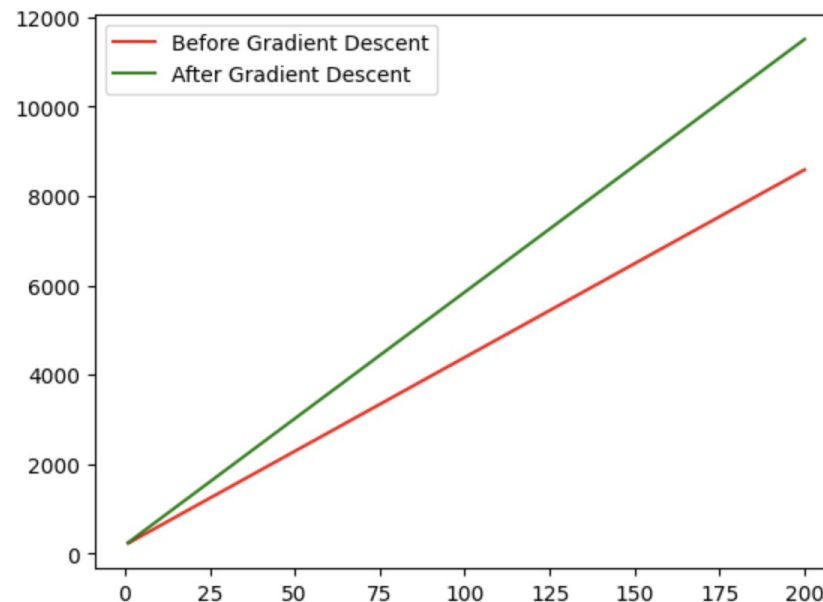


Linear Regression Example

We can now visualize the initial and

```
x_axis = np.arange(1,200,0.01)
y_initial = initial_a0 + initial_a1 * x_axis
y_final = a0 + a1 * x_axis

plt.plot(x_axis, y_initial, 'r', label = 'Before Gradient Descent')
plt.plot(x_axis, y_final, 'g', label = 'After Gradient Descent' )
plt.legend()
plt.show()
```

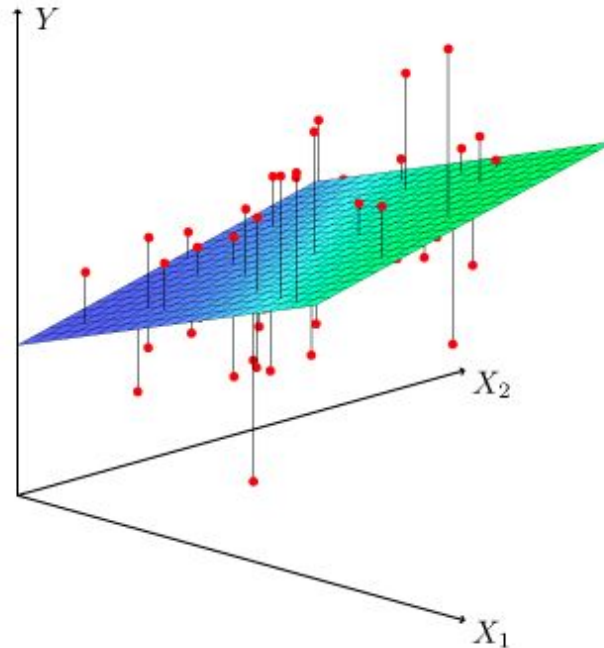


Lab: Linear Regression and Regression Error Metrics

Multi-Linear Regression

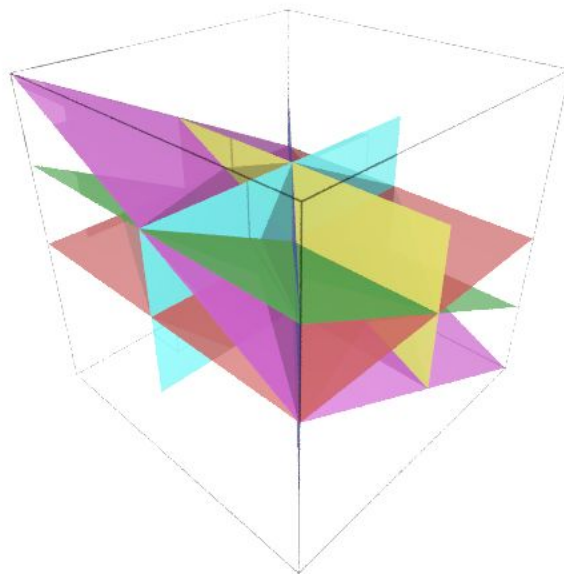
Multi-Linear Regression

While we can model the relationship between a feature and the output using a line, we can model the relationship between two input feature and the output using a plane.



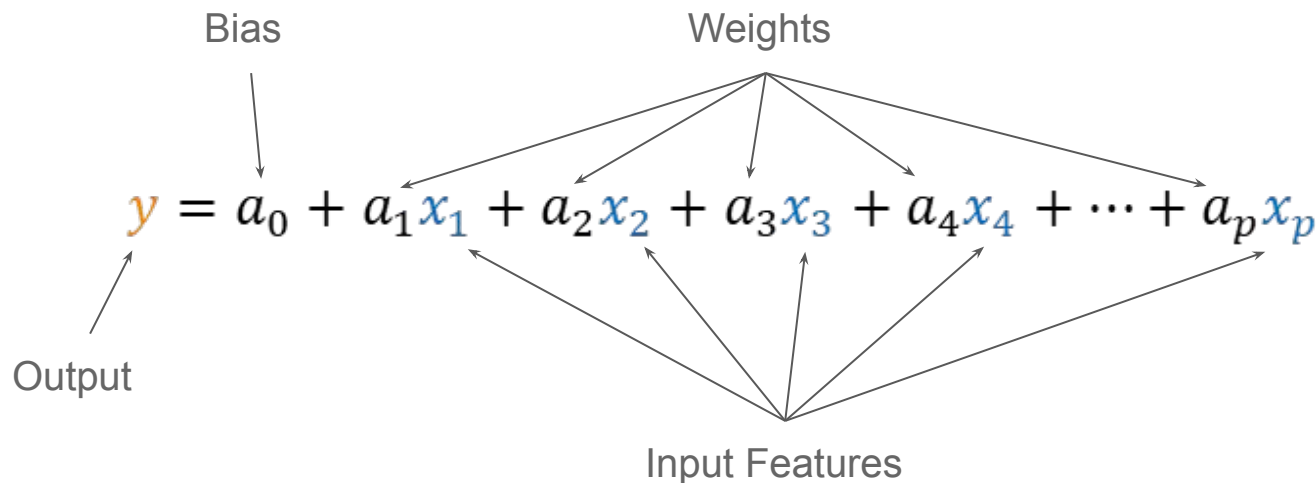
Multi-Linear Regression

Using the same logic, the relationship between three or more features and the output can be modeled using a hyperplane.



Multi-Linear Regression

Thus, multi-linear regression is modeled by a **polynomial**. The number of variables in the polynomial depends on the number of input features.



Updates in multi-linear regression

We apply the gradient descent update rule for each weight in the multi-linear regression model **in each iteration**.

Derivative of the cost function: $\frac{\partial Cost}{\partial a_0}, \frac{\partial Cost}{\partial a_1}, \frac{\partial Cost}{\partial a_2}, \dots, \frac{\partial Cost}{\partial a_n}$

Update rule: $a_i := a_i - learning_rate * \frac{\partial Cost}{\partial a_i}$

Classification Algorithms

Binary vs Multi-Class Classification

Choose the appropriate classification type for your data analysis



Binary Classification

Focus on two outcomes



Multiclass Classification

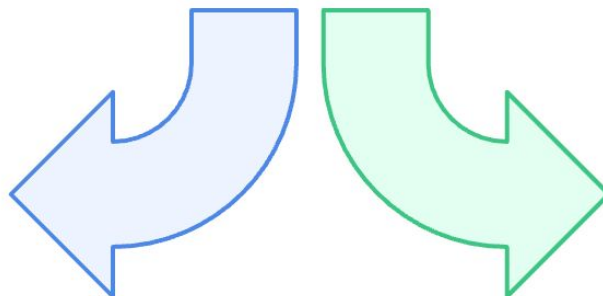
Handle multiple classes

Binary vs Multi-Class Classification

Which classification method to use?

Binary Classification

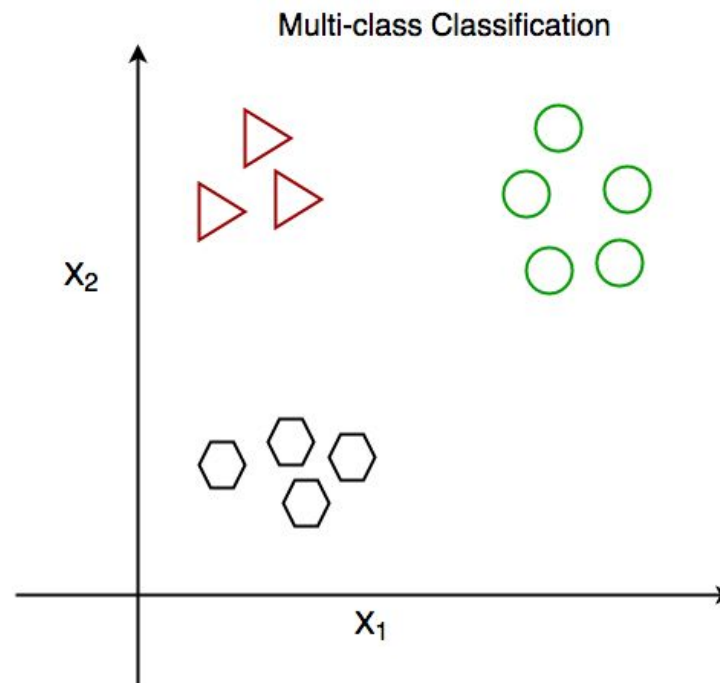
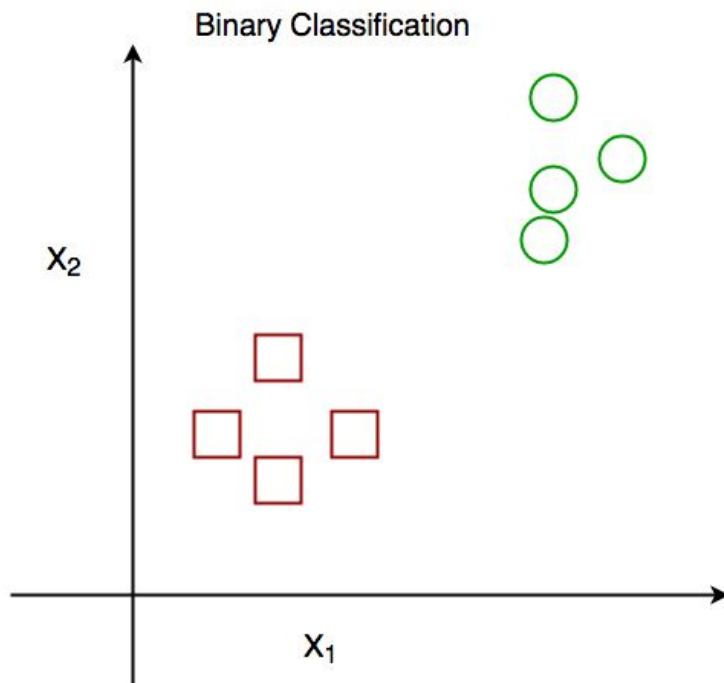
Suitable for tasks with two distinct outcomes, simplifying decision-making.



Multiclass Classification

Appropriate for tasks with multiple outcomes, adding complexity.

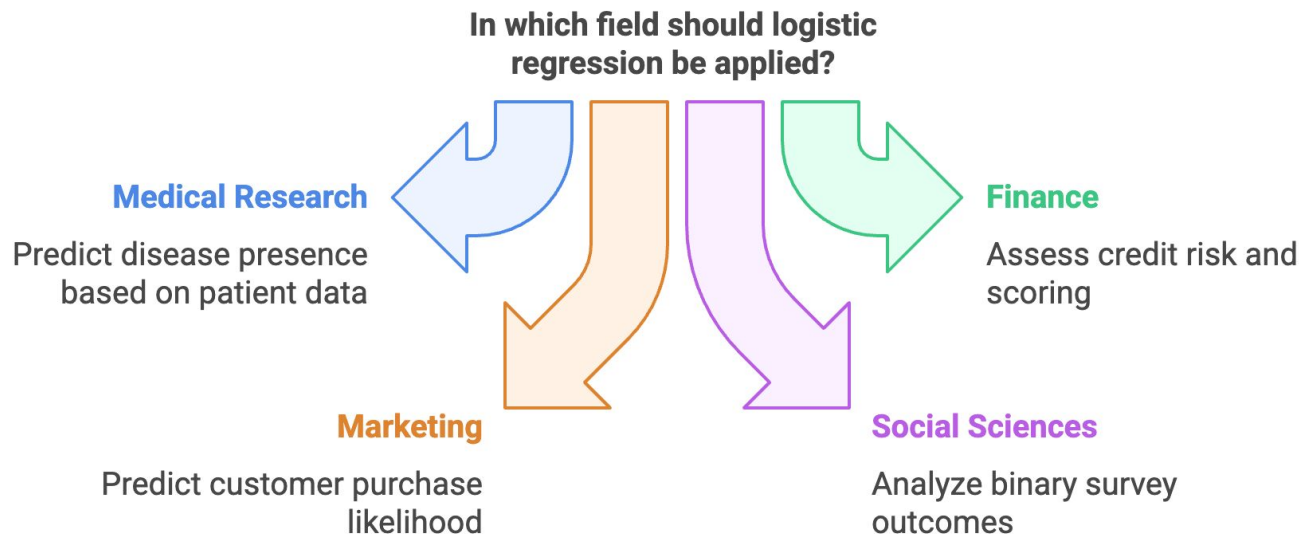
What would the data look like?



Logistic Regression

Logistic Regression

Logistic Regression is a statistical method used for **binary classification** problems, where the outcome variable is categorical with two possible outcomes.



Logistic Regression

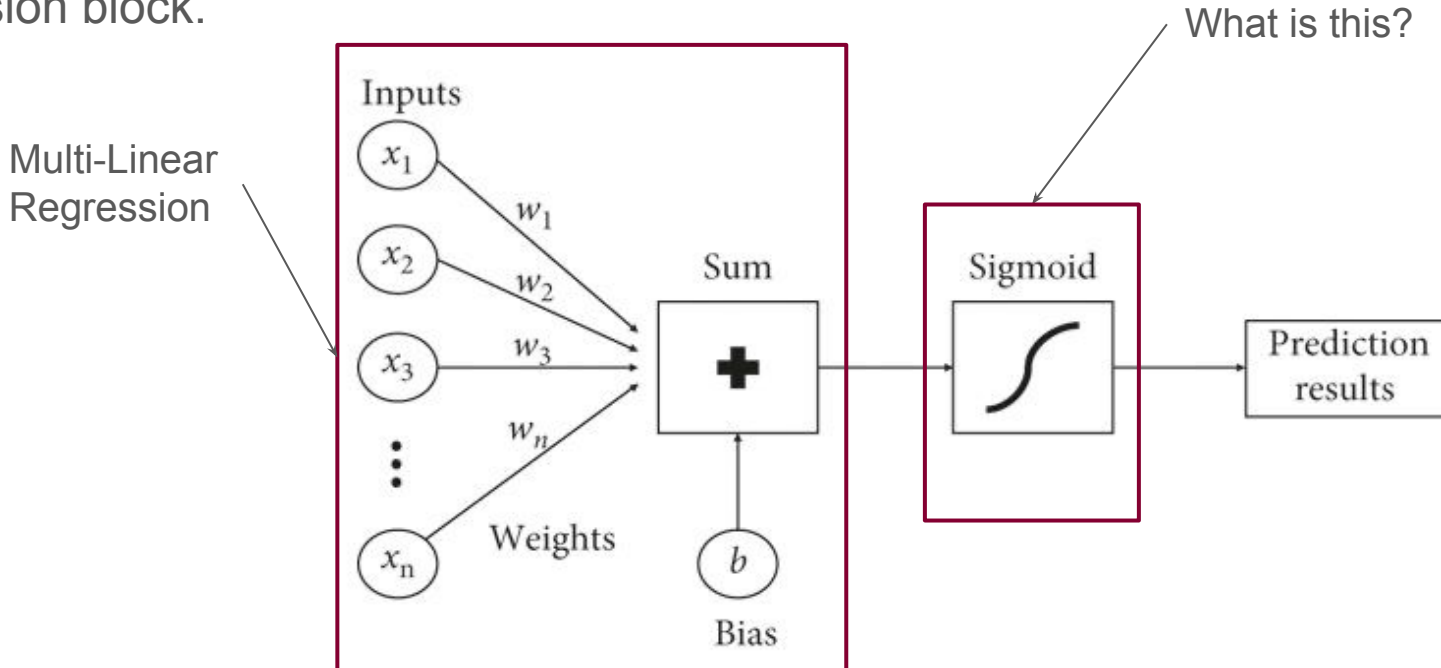
So why does it have 'Regression' in its name if it is a classification problem?

Logistic Regression is an **extension of linear regression** and multi-linear regression to be used for classification.

This extension is done by adding a **mapping function** that would allow us to map the output of the linear regression part to a class.

Logistic Regression

The equation of logistic regression: We add a “Sigmoid” function after the linear regression block.

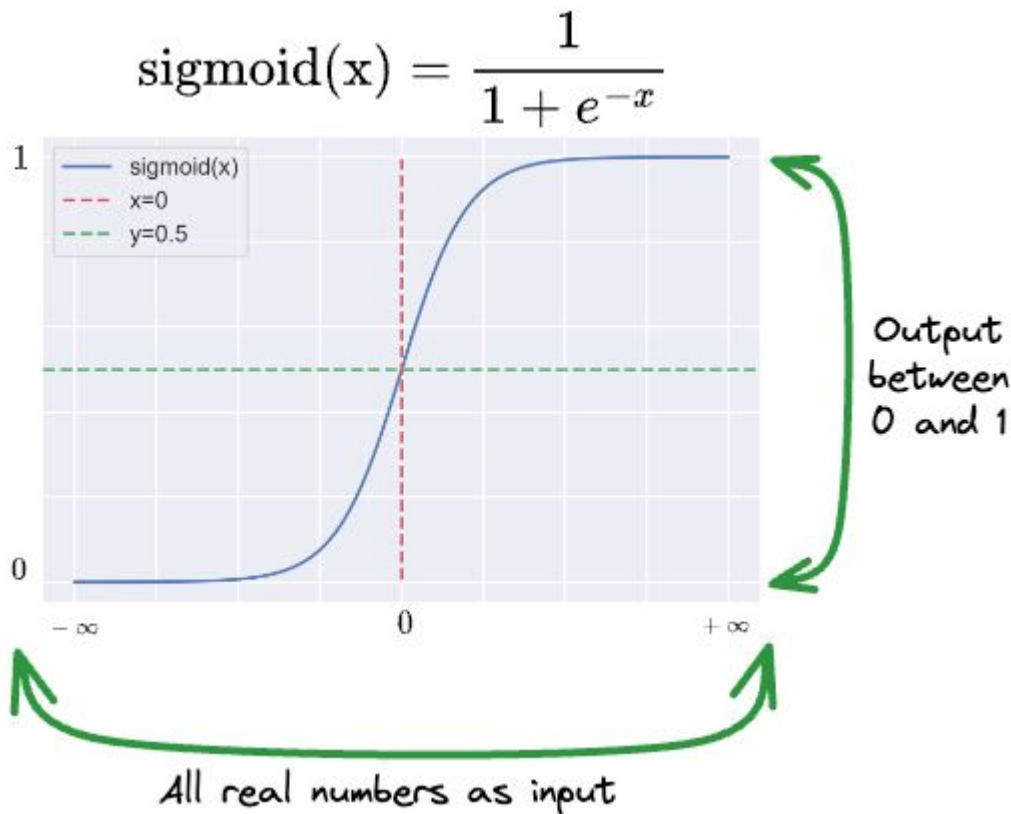


Logistic Regression

The sigmoid function:

Given that our threshold is 0.5: If the output of the sigmoid function is above 0.5, the data point belongs to one class.

Otherwise, the data point belongs to class B.

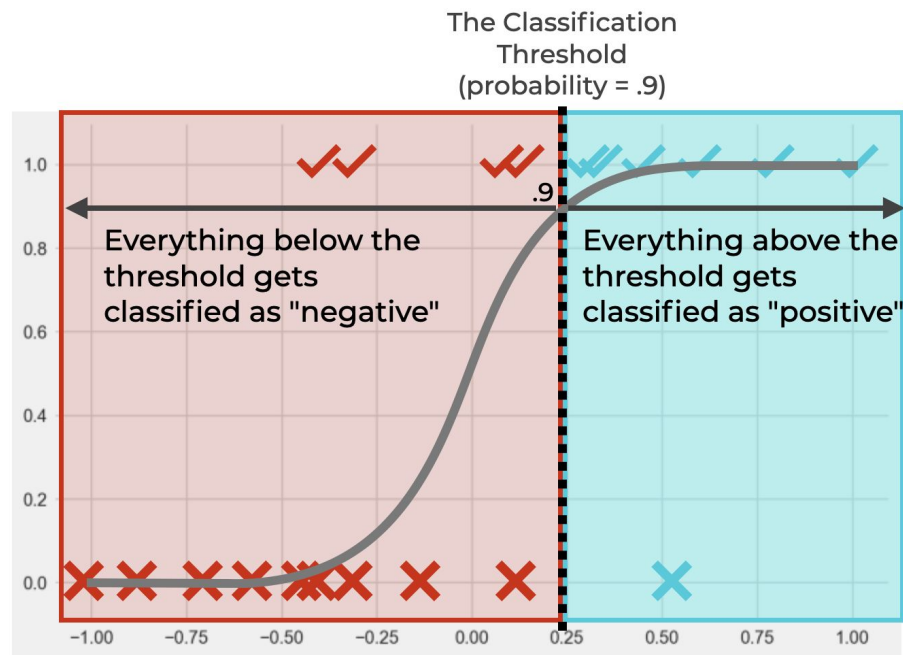


Logistic Regression

Should the threshold be always be 0.5? No.

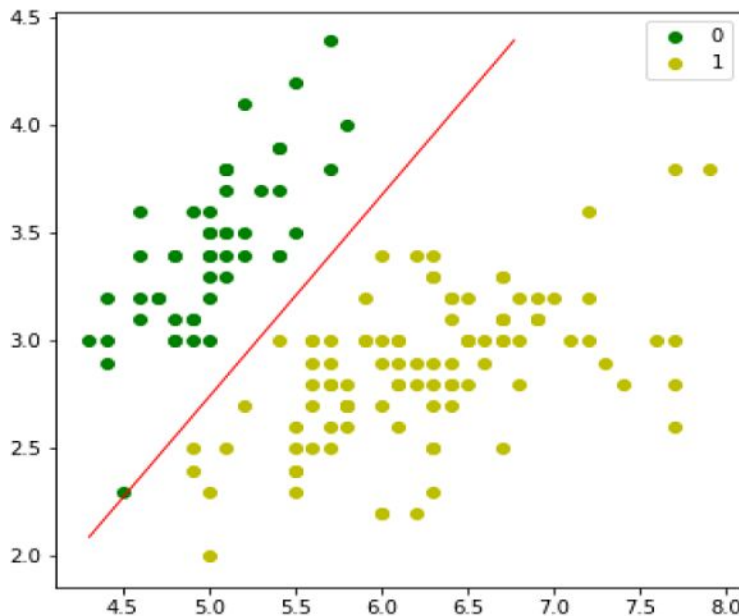
This is the default threshold.
However, according to your data,
you might find other threshold more
suitable.

CHANGING THE THRESHOLD CHANGES HOW EXAMPLES GET CLASSIFIED



What would the line in LR block represent?

We saw that the sigmoid function is preceded by a linear regression block. Instead of modeling the data, the line is used as a **classifying boundary**.



Trainable parameters in Logistic Regression

We know what are the trainable parameters in the (multi) linear regression block (coefficients + bias).

Do we have any trainable parameters in the sigmoid block?

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

Trainable parameters in Logistic Regression

What about the threshold that we use to decide if the output of the sigmoid block maps to class A or class B?

The threshold is pre-set and not changed or “learned” during the training process.
This means that it is a fixed parameter and not a trainable parameter.
We call these types of parameters “**hyper-parameters**”.

Training Classification Algorithms

Classification Training Lifecycle

To update the parameters of a classification function, we will have to follow the same training steps as before:

1. Random initialization of training parameters
2. Passing the function through one or multiple training samples
3. Calculating the error
4. Using gradient descent to update the value of the trainable parameters

Which part do you think differs between classification and regression?

Classification Training Lifecycle

To update the parameters of a classification function, we will have to follow the same training steps as before:

1. Random initialization of training parameters
2. Passing the function through one or multiple training samples
3. Calculating the error
4. Using gradient descent to update the value of the trainable parameters

We cannot measure the error in using the same metrics between classification and regression.



Classification Error Metrics- Binary

Let's say we have this example where we will predict if a person is male (1) or female (0) based on their height

Height	Gender
160	0
170	1
163	0
175	1

Features



Target (0 or 1)



Classification Error Metrics - Binary

The output of the logistic regression function will be the output of the sigmoid function. This means that it will be a float between 0 and 1.

Height	Gender	G_Pred
160	0	0.8
170	1	0.2
163	0	0.7
175	1	0.1

Binary Cross Entropy

The goal of training is to maximize the likelihood that the model assigns to the correct labels. Instead of maximizing likelihood directly, we minimize the negative log-likelihood, leading to:

$$L(y_i, y'_i) = -y_i \log(y'_i) - (1 - y_i) \log(1 - y'_i)$$

Diagram illustrating the components of the Binary Cross Entropy Loss Function:

- Loss Function:** Indicated by a purple arrow pointing to $L(y_i, y'_i)$.
- Real Value:** Indicated by a blue arrow pointing to y_i .
- Predicted Value:** Indicated by a green arrow pointing to y'_i .

Binary Cross Entropy

This function penalizes wrong predictions exponentially by taking the log of predicted probabilities. Logarithmic scaling ensures that confident wrong predictions are penalized much more than weakly wrong predictions.

True Label	Predicted Probability	Contribution to Log Loss
1	0.99	Small loss (~ 0.01)
1	0.50	Moderate loss (~ 0.69)
1	0.01	Huge loss (~ 4.61)

Binary Cross Entropy

Means that it is used to calculate the error and update binary classification problems.

- Cross-entropy measures the distance between two probability distributions:
 - True distribution P (actual labels)
 - Predicted Distribution Q (model's predictions)

The loss function we saw is called **Binary Cross Entropy**.

Entropy measures the uncertainty or unpredictability of a probability distribution. If an event is certain, entropy is low

Binary Cross Entropy

Imagine we have two models predicting the probability of an image being a "cat".

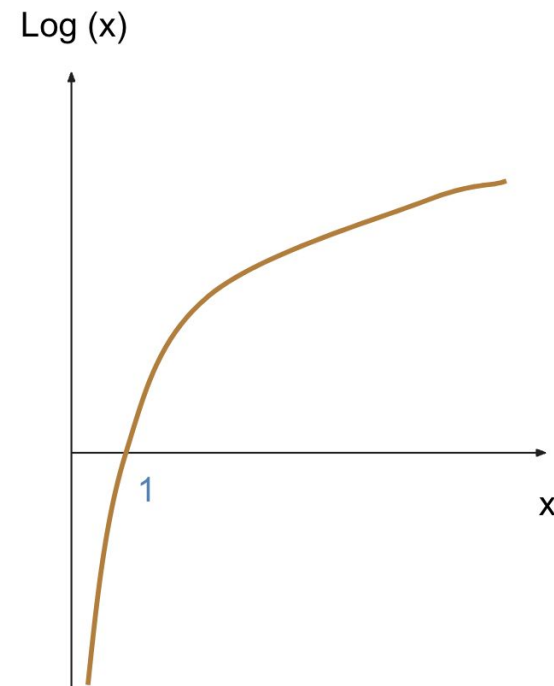
Model	$P(\text{cat})$	$P(\text{dog})$	Cross-Entropy Loss
Model A (correct confident prediction)	0.95	0.05	Low
Model B (incorrect confident prediction)	0.05	0.95	High

Binary Cross Entropy

$$L(y_i, y'_i) = -y_i \log(y'_i) - (1 - y_i) \log(1 - y'_i)$$

Real Value	Predicted Value	Loss or Error
0	0.2	0.09
0	0.8	0.7
1	0.1	1
1	0.9	0.045

Note: The *log* is computed with a base 10.



Binary Cross Entropy: Loss vs Cost

As we mentioned before, a loss (or error) is for one data sample while cost is for multiple data samples.

$$L(y_i, y'_i) = -y_i \log(y'_i) - (1 - y_i) \log(1 - y'_i)$$

$$C(y_i, y'_i) = \frac{-1}{n} \sum_{i=1}^n y_i \log(y'_i) + (1 - y_i) \log(1 - y'_i)$$

Classification Error Metrics

So far, with logistic regression, we saw a binary classification model that outputs a value between 0 and 1 where:

- Anything below the threshold (0.5) belongs to one class and anything above belongs to another.

What if we have multiple classes?

Multi-Class Classification

Multi-class classification models output an **array of probabilities** instead of one probability (likelihood of belonging to class A).

Feature	Binary Classification (BCE)	Multi-Class Classification (CCE)
Number of Classes	2 (e.g., Cat vs. Dog)	3+ (e.g., Cat, Dog, Rabbit)
Model Output	Single probability value (for one class)	Probability distribution over all classes
Target Representation	Single value (0 or 1)	One-hot encoded vector
Example True Labels	<code>[0]</code> (not a cat) OR <code>[1]</code> (cat)	<code>[1, 0, 0]</code> (cat) OR <code>[0, 1, 0]</code> (dog) OR <code>[0, 0, 1]</code> (rabbit)
Example Model Predictions	<code>[0.85]</code> (85% cat, 15% not cat)	<code>[0.70, 0.20, 0.10]</code> (70% cat, 20% dog, 10% rabbit)

Classification Error Metrics

For multi-class classification, we use **Categorical Cross Entropy** instead of Binary Cross Entropy.

$$L = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k y_{ij} \log(\hat{y}_{ij})$$

k is the number of classes.

Predicted value
sample i belongs to
class j

True value sample i
belongs to class j

We sum over all classes for each sample since one-hot encoding ensures only the true class contributes to the loss.



Categorical Cross Entropy

Feature	Binary Classification (BCE)	Multi-Class Classification (CCE)
Number of Classes	2 (e.g., Cat vs. Dog)	3+ (e.g., Cat, Dog, Rabbit)
Model Output	Single probability value (for one class)	Probability distribution over all classes
Cross-Entropy Loss Calculation	$-y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$	$-\sum y_i \log(\hat{y}_i)$ (only for the correct class)
Loss Example (if true label is "Cat")	$-1 \log(0.85) = 0.162$	$-1 \log(0.70) = 0.357$
Decision Rule	If $P(y = 1) \geq 0.5$, predict class 1, else class 0	Choose the class with the highest probability

Logistic Regression for MultiClass Classification

Can we use logistic regression for multiclass classification? Yes.

To do so, we have two options:

1. One vs All algorithm. We train multiple binary logistic regression models, each distinguishing one class from all others.
2. If we switch the sigmoid function with a softmax function, we can output an array of probabilities instead of a singular probability value.

Formula for softmax function

$$s(x_i) = \frac{\overset{\textcircled{1} \text{Exponent of } x_i}{e^{x_i}}}{\underbrace{\sum_{j=1}^n e^{x_j}}_{\textcircled{2} \text{Total value of } x \text{ exponent}}}$$

softmax is a simple formula that divides **(1) Exponent of xi** by **(2) Total value of x exponent**.

Gradient Descent using BCE and CCE

We perform the same steps in the gradient descent update where we also derive the cost function to insert it in the gradient update rule

$$C(y_i, y'_i) = \frac{-1}{n} \sum_{i=1}^n y_i \log(\sigma(a_0 + a_1 x_i)) + (1 - y_i) \log(1 - (\sigma(a_0 + a_1 x_i)))$$

$$\frac{\partial C}{\partial a_0} = -\frac{1}{n} \sum_{i=1}^n [y_i - \sigma(a_0 + a_1 x_i)]$$

Training logistic regression from scratch in lab 'Regression Lab'

Naive Bayes

Naive Bayes

Naive Bayes is a probability-based algorithm. In logistic regression, which was a linear-regression based algorithm that is used to output a probability.

However, in Naive Bayes, we use a **probability function** to calculate the **probability of a data point belonging to a class**.

Probability Recap

Calculating the probability of an event is to find how much likely this event is to happen.

$$P(event) = \frac{\text{number of occurrences of that event}}{\text{Total number of trials}}$$

Probabilities of calibrated dice:

Event	1	2	3	4	5	6
Probability	1/6	1/6	1/6	1/6	1/6	1/6

Calibrated Dice = Equal Probabilities

Table reflecting the probability of obtaining a specific number when rolling the dice.



Probability Recap

Expectation: Summation of all possible values of a random variable, multiplied by the probability of each.

$$E(X) = \sum_{i=1}^n x_i p_i$$

Random Variable $\rightarrow$ $E(X)$

Values taken by the random variable $\rightarrow$ x_i

Probability associated with each value $\rightarrow$ p_i

x	1	2	3	4	5	6
p	1/6	1/6	1/6	1/6	1/6	1/6

$$E(X) = \left(1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + 3 \cdot \frac{1}{6} + 4 \cdot \frac{1}{6} + 5 \cdot \frac{1}{6} + 6 \cdot \frac{1}{6} \right) = 3.5$$



Probability Recap

In most cases, the probability is not equally distributed.

- What if we don't know if the dice is calibrated?
- What if we don't know if the coin is calibrated?
- Then we don't know the probabilities, we need to **approximate them**.

Event	1	2	3	4	5	6
Probability	?	?	?	?	?	?



Naive Bayes

The Naive bayes algorithm is based on the Bayes Theorem for conditional probability.

Let's remember conditional probability:

$$P(A/B) = \frac{P(A \cap B)}{P(B)}$$

$$P(A \cap B) = P(B \cap A)$$

$$P(B/A) = \frac{P(B \cap A)}{P(A)}$$

$$P(B \cap A) = P(B/A) \cdot P(A)$$

$$P(A/B) = \frac{P(B/A) \cdot P(A)}{P(B)}$$

Naive Bayes

How does conditional probability apply to prediction problems?

In our case of classification, we want to calculate the probability of a data point belonging to class A, to class B, to class C... Then, we will use the highest probability to select the class.

Naive Bayes

What guides our decision to choosing the class? The information in the features.

$$P(A/B) = \frac{P(B/A).P(A)}{P(B)}$$

$$P(Class/Features) = \frac{P(Features/Class).P(Class)}{P(Features)}$$



Posterior Probability

Likelihood of the Evidence given that the Hypothesis is True

Prior Probability of the Hypothesis

$$P(H|E) = \frac{P(E|H) * P(H)}{P(E)}$$

Posterior Probability of the Hypothesis given that the Evidence is True

Prior Probability that the evidence is True

A diagram illustrating the components of the posterior probability formula. The formula is $P(H|E) = \frac{P(E|H) * P(H)}{P(E)}$. Four text labels are connected to the formula by blue arrows: 'Likelihood of the Evidence given that the Hypothesis is True' points to $P(E|H)$; 'Prior Probability of the Hypothesis' points to $P(H)$; 'Posterior Probability of the Hypothesis given that the Evidence is True' points to $P(H|E)$; and 'Prior Probability that the evidence is True' points to $P(E)$.

Naive Bayes Training

Naïve Bayes simplifies training by summarizing data into probability values instead of storing all examples.

During training, it calculates:

- **Class priors:** $P(C_k)$ – the probability of each class.
- **Feature likelihoods:** $P(X|C_k)$ – how often a feature appears for a given class.

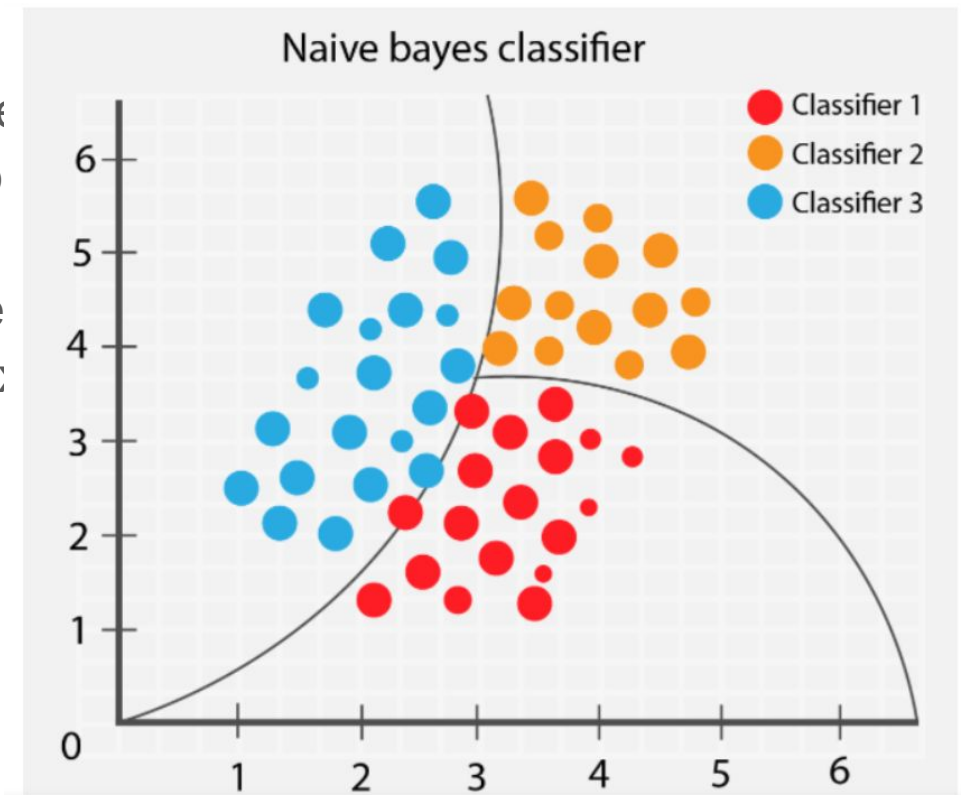
For different data types:

- **Categorical data:** Counts feature occurrences per class and converts them into probabilities.
- **Continuous data:** Estimates the mean and variance of each feature per class.

Naive Bayes Prediction

After calculating class priors and feature
Theorem to determine posterior probab

This approach is computationally efficie
probability calculations, without comple



Naive Bayes Training Summary

Naïve Bayes training involves three simple steps:

1. Count feature occurrences for each class.
2. Estimate likelihoods using probability distributions (e.g., Gaussian for continuous data, Multinomial for text).
3. Apply Bayes' Theorem to calculate posterior probabilities.



Naive Bayes Example

To solidify how Naïve Bayes functions, we will consider an example with one feature.

The question we want to answer: Kids will play if the weather is sunny?

How do we start with applying Naive Bayes?

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes
Overcast	Yes
Rainy	No



Naive Bayes Example

Step 1: Count feature occurrences per class.

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes
Overcast	Yes
Rainy	No

Frequency Table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
Grand Total	5	9

Naive Bayes Example

Step 2: Calculate the class probabilities and feature likelihoods.

In order to calculate the likelihood of kids playing with respect to the weather condition, we begin by computing the probability of each condition and event.

$$P(\text{Rainy}) = \frac{\text{Number of occurrences of rainy games}}{\text{Total number of occurrences}} = \frac{3 + 2}{14} = \frac{5}{14}$$

Frequency Table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
Grand Total	5	9

Likelihood table				
Weather	No	Yes		
Overcast		4	=4/14	0.29
Rainy	3	2	=5/14	0.36
Sunny	2	3	=5/14	0.36
All	5	9		
	=5/14	=9/14		
	0.36	0.64		

Naive Bayes Example

Step 3: Apply Bayes' Theorem to compute posterior probabilities.

The model assigns the class with the highest probability

$$P(\text{YES}/\text{SUNNY}) = \frac{P(\text{SUNNY}/\text{YES}) \cdot P(\text{YES})}{P(\text{SUNNY})} = 0.6$$

Annotations for the equation above:

- $\frac{3}{9}$ points to $P(\text{SUNNY}/\text{YES})$
- $\frac{9}{14}$ points to $P(\text{YES})$
- $\frac{5}{14}$ points to $P(\text{SUNNY})$

Frequency Table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
Grand Total	5	9

Likelihood table				
Weather	No	Yes		
Overcast		4	=4/14	0.29
Rainy	3	2	=5/14	0.36
Sunny	2	3	=5/14	0.36
All	5	9		
	=5/14	=9/14		
	0.36	0.64		

Naive Bayes Assumptions

We notice that we calculate the feature likelihoods without taking into consideration more than one feature at a time.

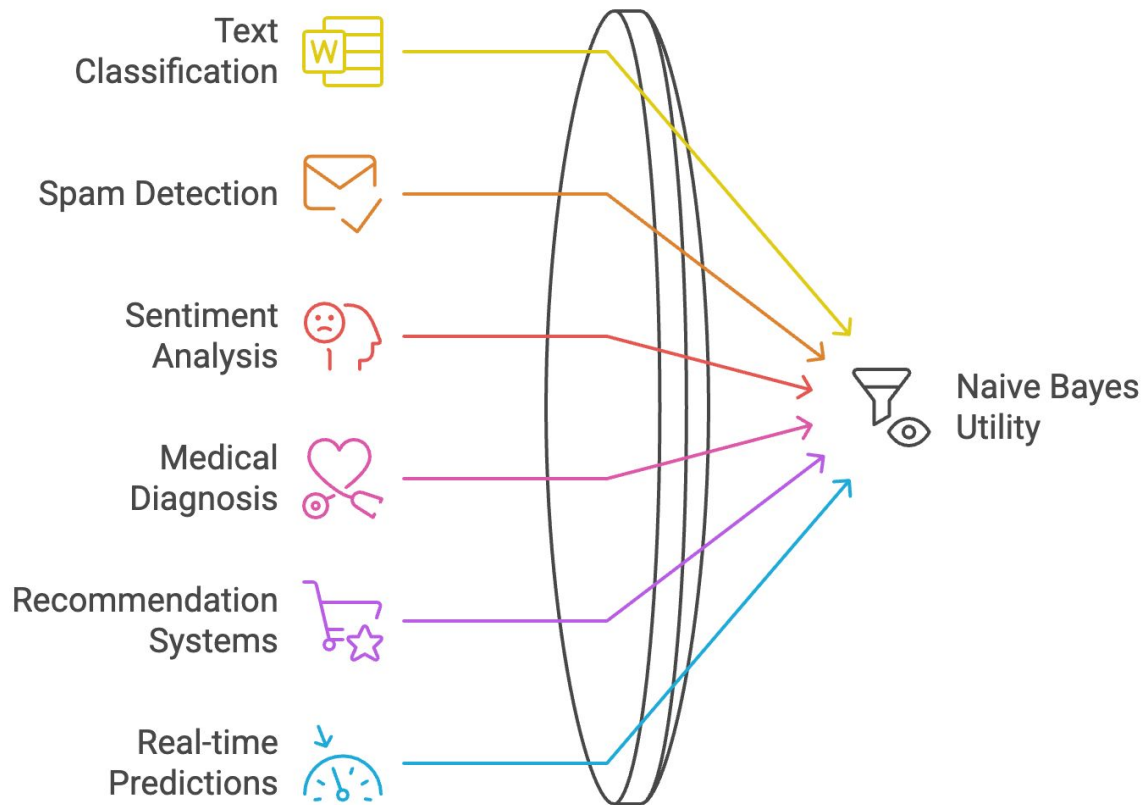
This means that **we assume there are no feature correlation or dependencies.**

This assumption may be false for a lot of use cases where there are at least minor correlation between the features that should be considered for accurate modeling.

$$P(X \cap Y) = P(X).P(Y)$$

$$P(X_1, X_2|Y) = P(X_1|Y).P(X_2|Y)$$

Applications of Naive Bayes



Training naive bayes in lab ‘Classification Algorithms’

Algorithms that can be used for both regression and classification

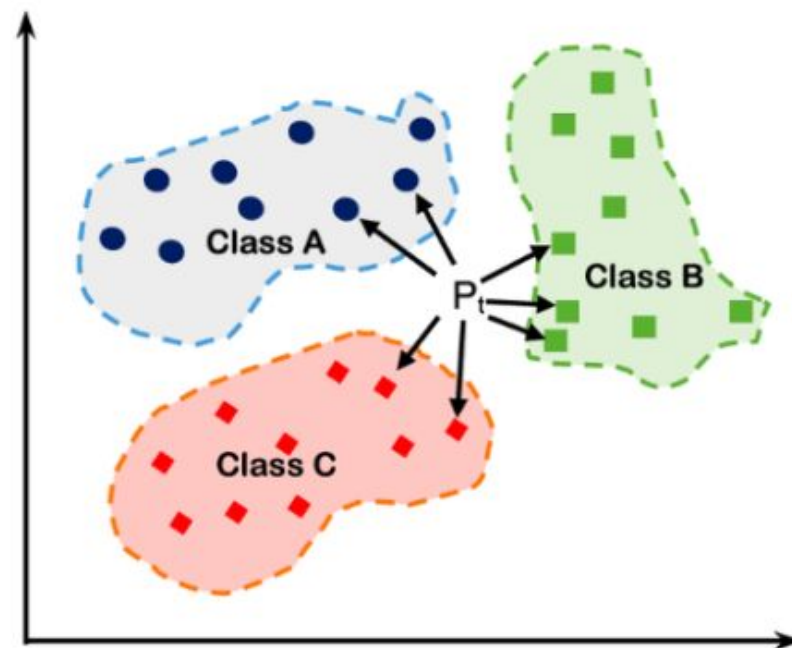
K-Nearest Neighbors For Classification

KNNs for Classification

It classifies a new data instances based on the k most similar training examples.

Similarity, in this case, is measured in distance.

K Nearest Neighbors





KNNs for Classification

How do KNNs choose which class the new data point belongs to?

K Nearest Neighbors

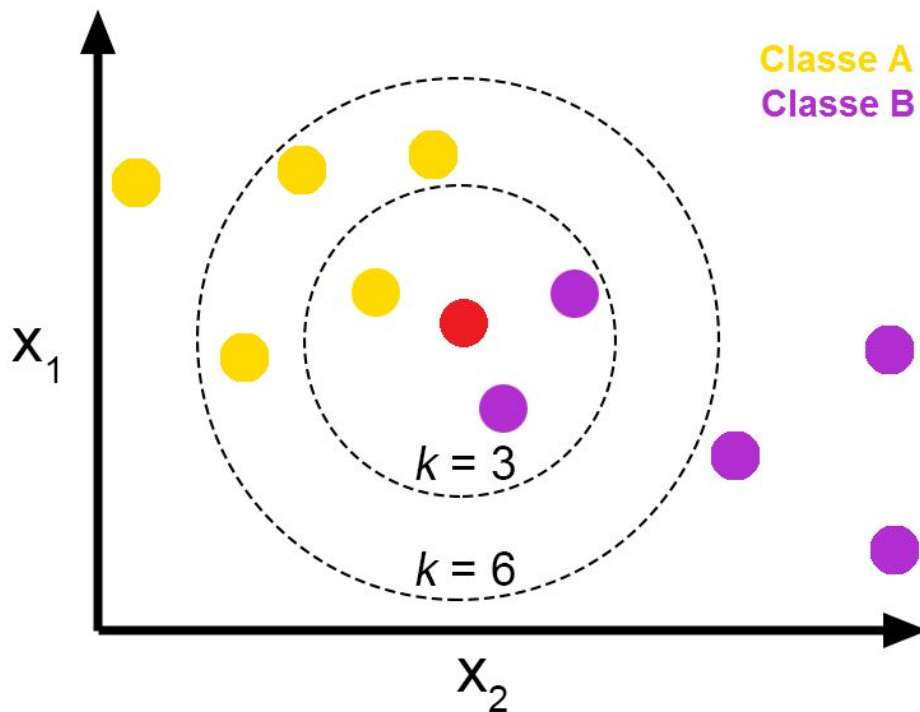
K is the number of nearest data points that surround the new input.

We obtain the K number of data points by measuring the distance between the data point and all the other data points. The labels of the K closest data points are used to decide which class the new data point belongs to.

KNNs for Classification

K is a hyperparameter that we set before the training process.

We use a majority voting mechanism to decide the class of the new data sample.



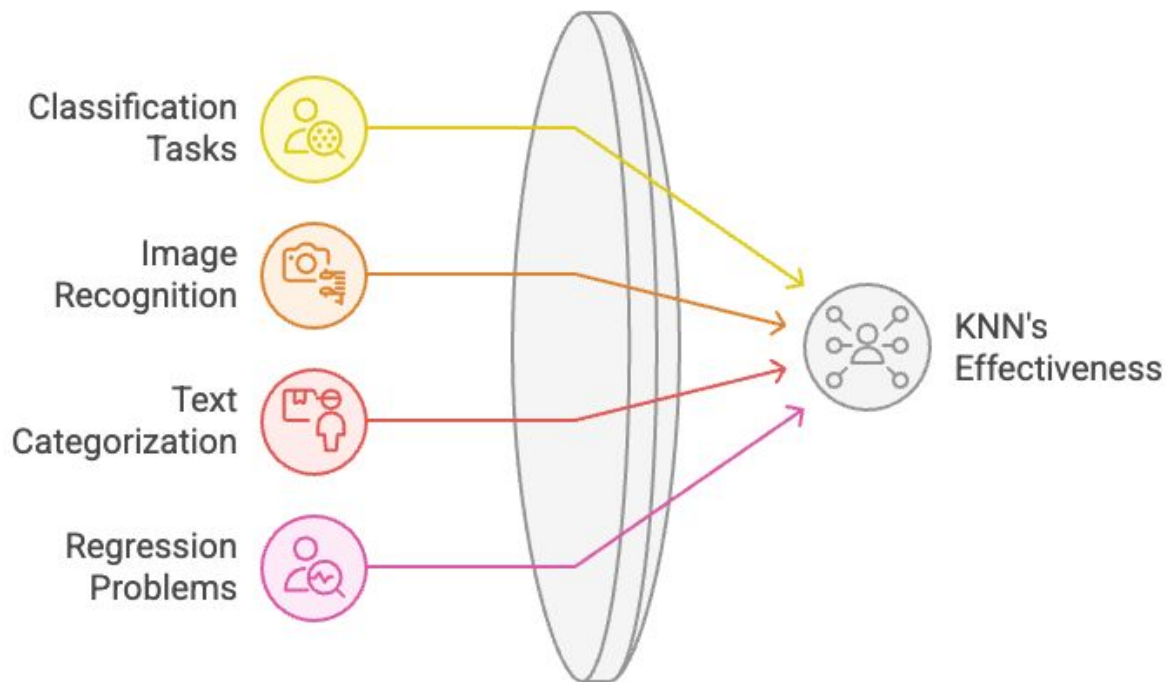
KNNs for Classification

What do KNNs learn? What are the trainable parameters? Are we just comparing every data point to the data points in our data set?

KNN is an **instance-based learning algorithm**, meaning that it **does not explicitly learn a function or a set of parameters during training**. Instead, it memorizes the training data and makes predictions by comparing new inputs to stored instances.

Learning in KNN is essentially **data storage and indexing**, which enables efficient nearest-neighbor searches.

KNNs for Classification



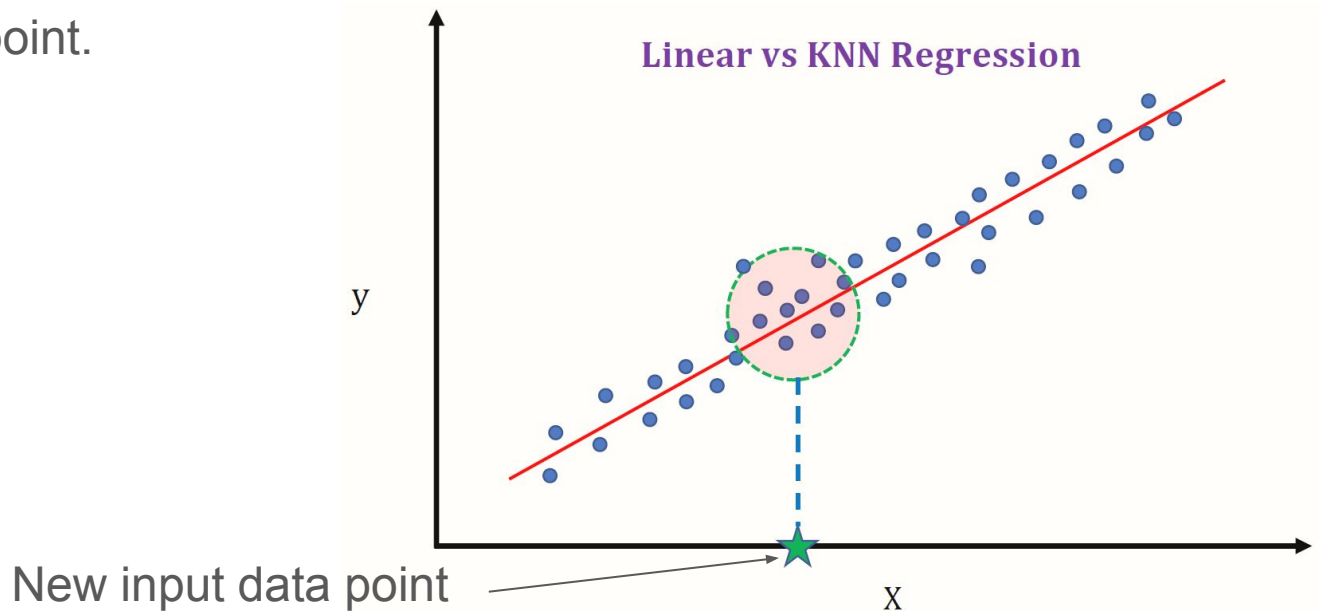
Training KNNs for classification in lab

‘Classification Algorithms’

KNNs for Regression

KNNs for Regression

Instead of using a majority vote to classify a new data point, KNNs for regression averages the value of the K nearest neighbors to estimate the value of the new data point.



KNNs for Regression

How KNN Regression Works:

1. **Distance Calculation:** For a new data point, the algorithm calculates the distance between this point and all points in the training dataset. Common distance metrics include Euclidean, Manhattan, and Minkowski distances.
2. **Identifying Neighbors:** The algorithm identifies the K data points in the training set that are closest to the new point based on the calculated distances.
3. **Prediction:** The target value for the new data point is predicted by averaging the target values of its K nearest neighbors.

Choice of K (Classification and Regression)

When K is too small (e.g., $K=1$ or 2)

1. The prediction is based on very few points, meaning small fluctuations in the data can have a big impact.
2. If there's noise in the dataset, the model might rely too much on those noisy points, leading to inconsistent or unstable predictions.

Think of it like asking just one or two people for advice—if they have extreme opinions, your decision might not be well-balanced.

Choice of K (Classification and Regression)

When K is too large (e.g., $K=50$ or 100)

- The prediction is influenced by many points, including ones that are farther away and might not be very similar.
- The model smooths out variations, which can make it less sensitive to specific details in the data.
- This is like averaging opinions from a very large group—while you get a general sense of the trend, you might lose important local nuances.

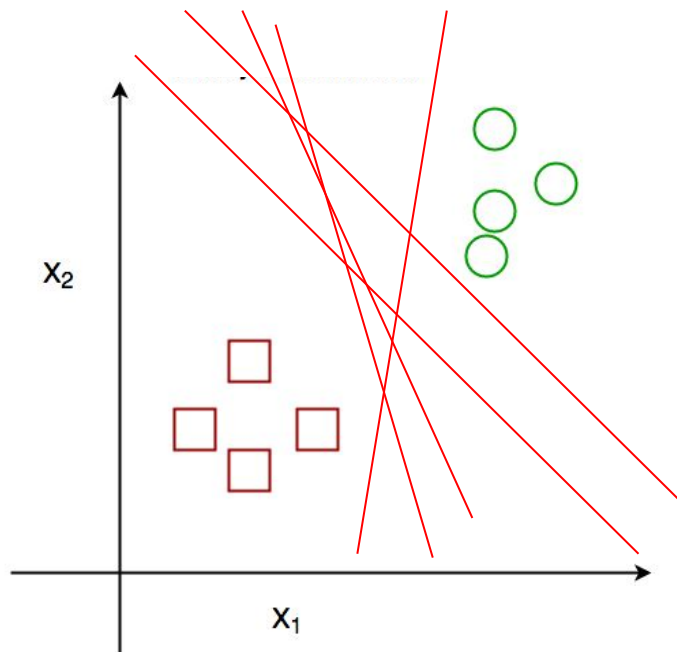
Training KNNs for Regression in lab 'Regression Lab'

Support Vector Machine for Classification

SVMs for classification

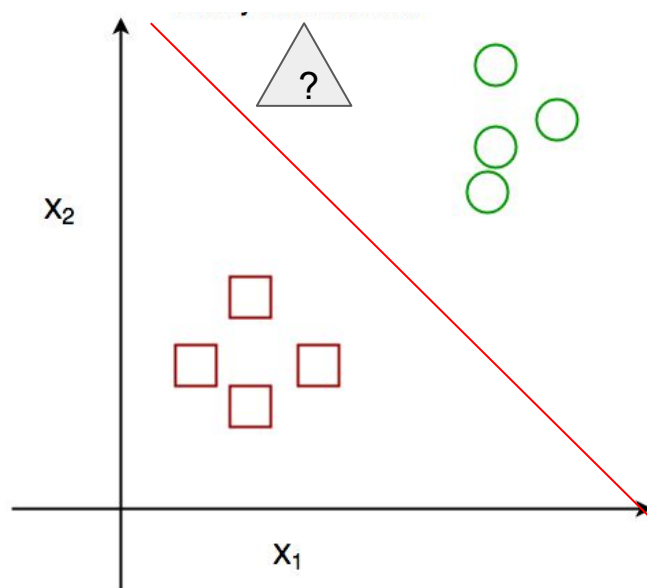
Suppose we have data to be used for binary classification.

What would you say makes the best linear separator of the two classes?



SVMs for classification

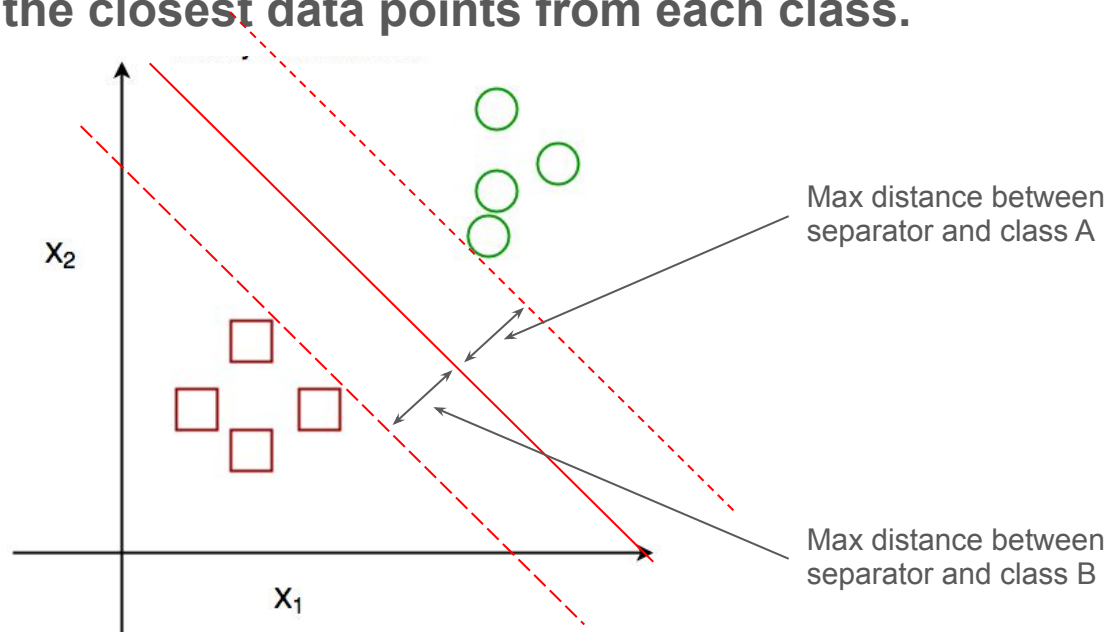
The goal of SVM is to curate the most accurate linear separator that will correctly classify any new input.



SVMs for classification

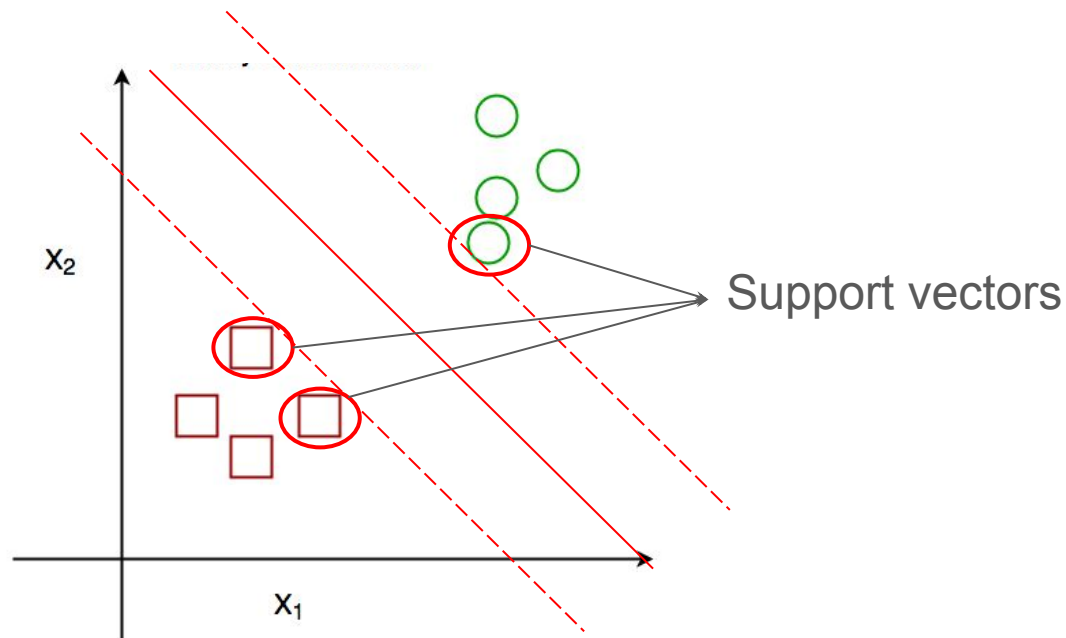
But what determines the best linear separator?

In support vector machine, the best linear separator is the line that **maximizes the distance between it and the closest data points from each class.**



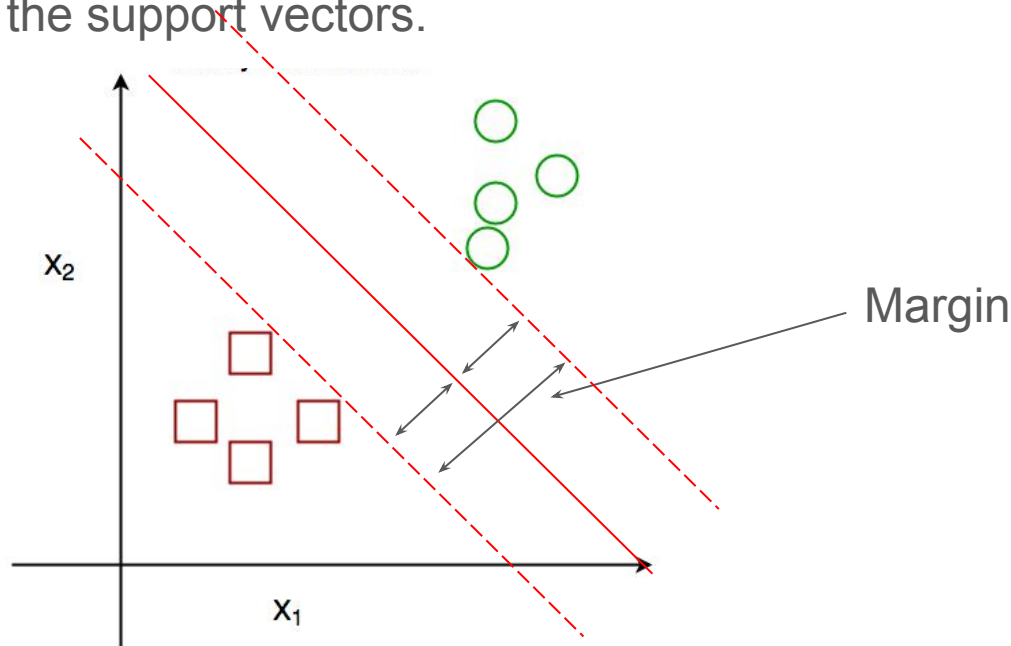
SVMs for classification

The closest data points to the separator are called the support vectors



SVMs for classification

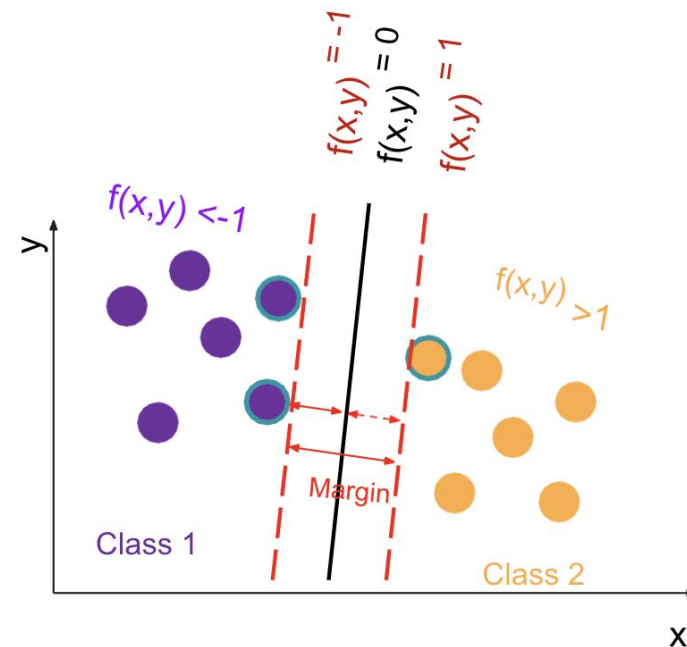
This distance, called the margin, should not only be maximized, but also equidistant between the support vectors.



SVM inference

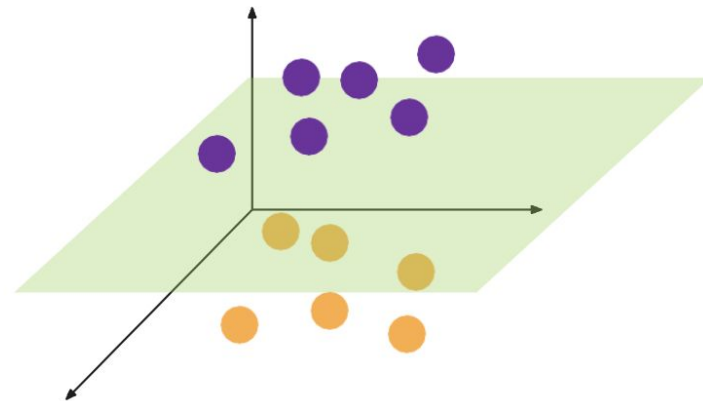
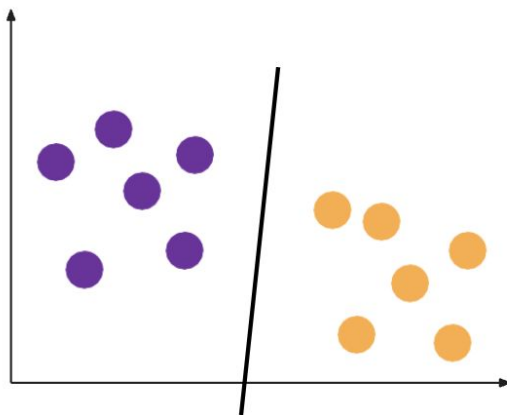
SVM classifies new data points by inserting the feature values in the equation of the line. Then, decides the class by checking the position of the new data point with respect to the line.

1. If $w_1x_1 + w_2x_2 + b > 0$. Then, the point falls above the line and it belongs to class A.
2. Otherwise, the point falls below the line and belongs to class B.



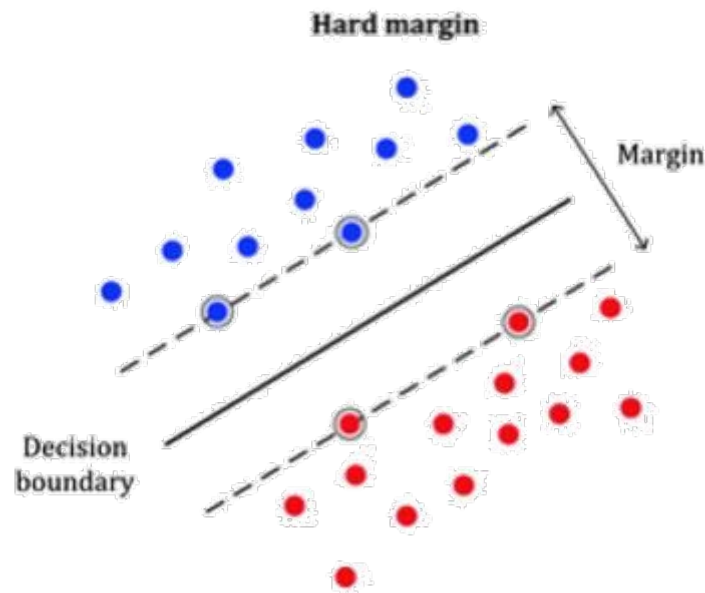
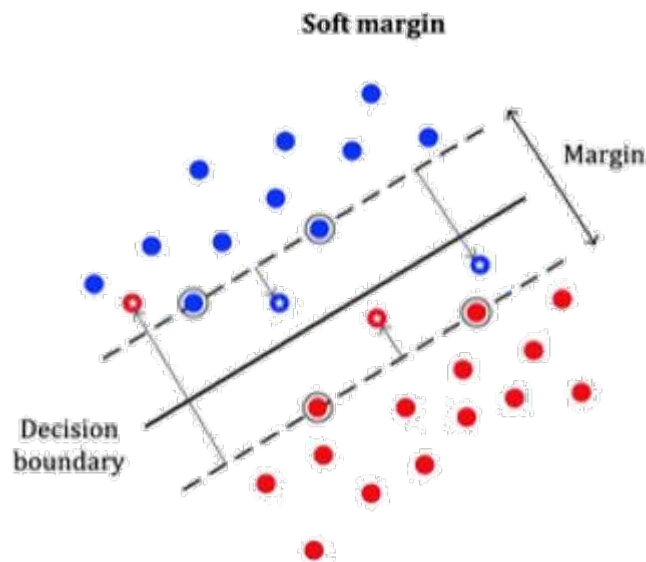
Higher Dimension SVM for Classification

If we had three features, the classifier becomes a classifier plane.



Types of Margins in SVMs

Data is not perfect. Some data points may lie within the margin. Thus, we have two types of margins:



SVM Hard Margin

Hard Margin conditions:

- The data should be linearly separable
- We select two parallel hyperplanes separating the two classes of data
- The hyperplanes are chosen so that the distance between them is as large as possible

SVM Soft Margin

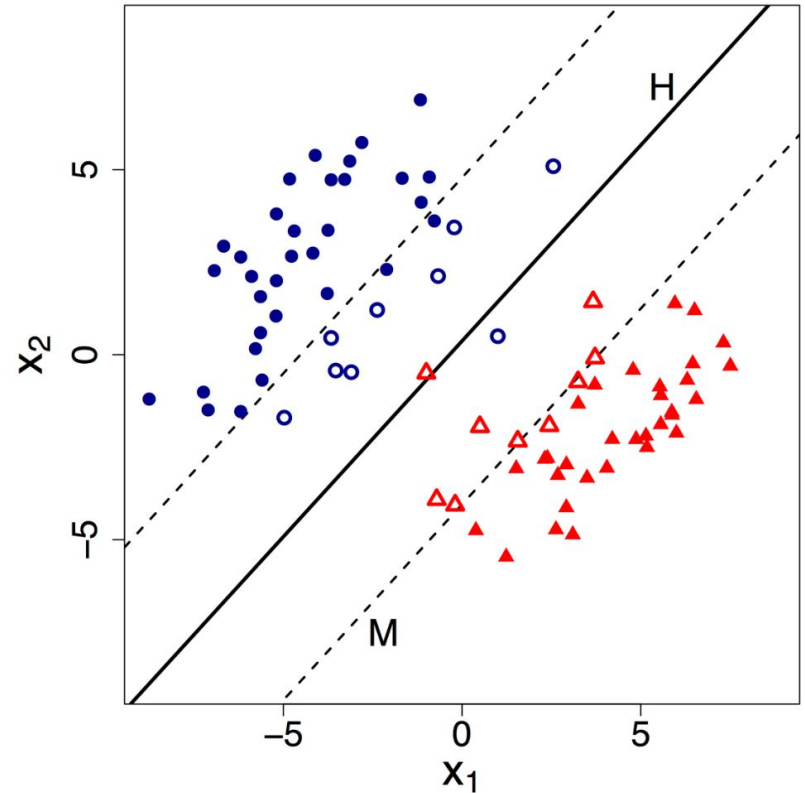
Soft Margin conditions:

- The data is not linearly separable
- We allow for errors in classification
- Finding the maximal margin means maximizing the margin between the data points and the hyperplane

SVM Error Function

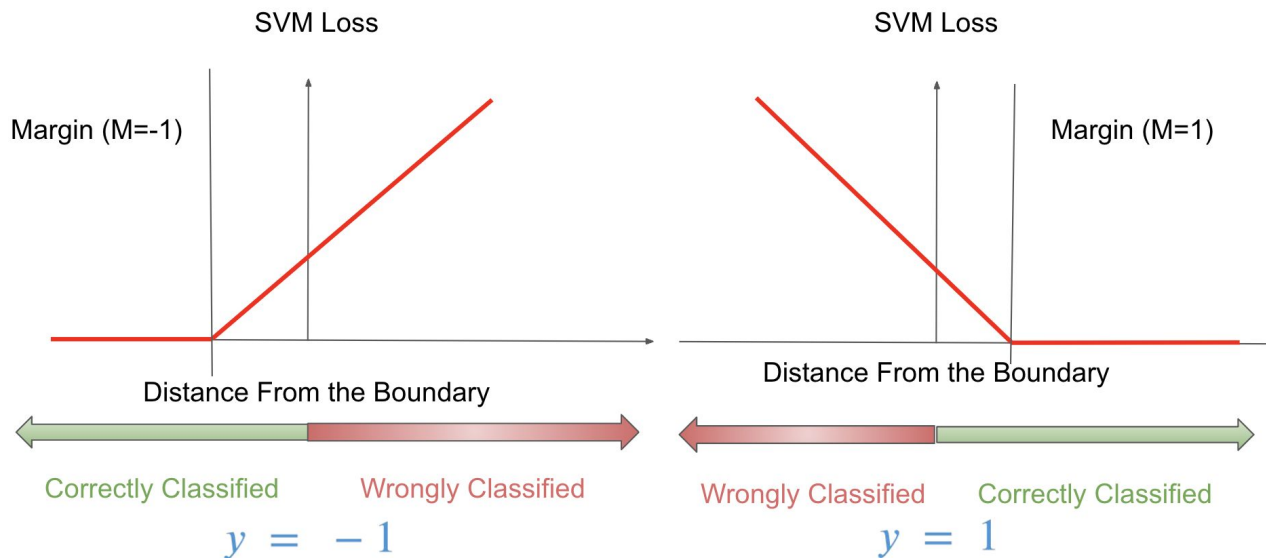
In the SVM training process, we need to penalize two scenarios:

1. Wrongly classified data points
2. Data points that are correctly classified but fall within the margin



SVM Error Function

Thus, we use a new loss function called **Hinge Loss** which considers the two penalties that we want.

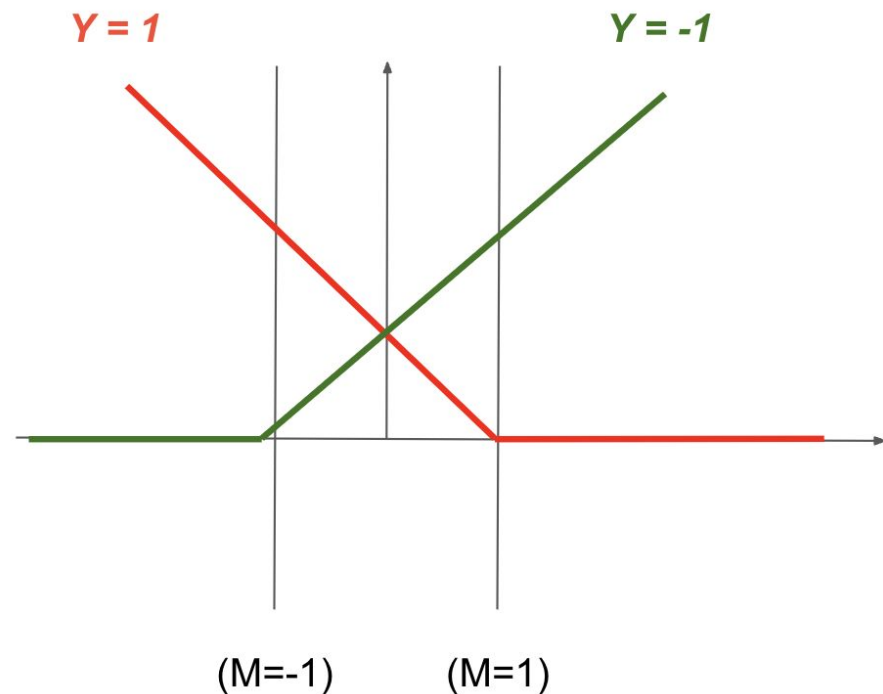




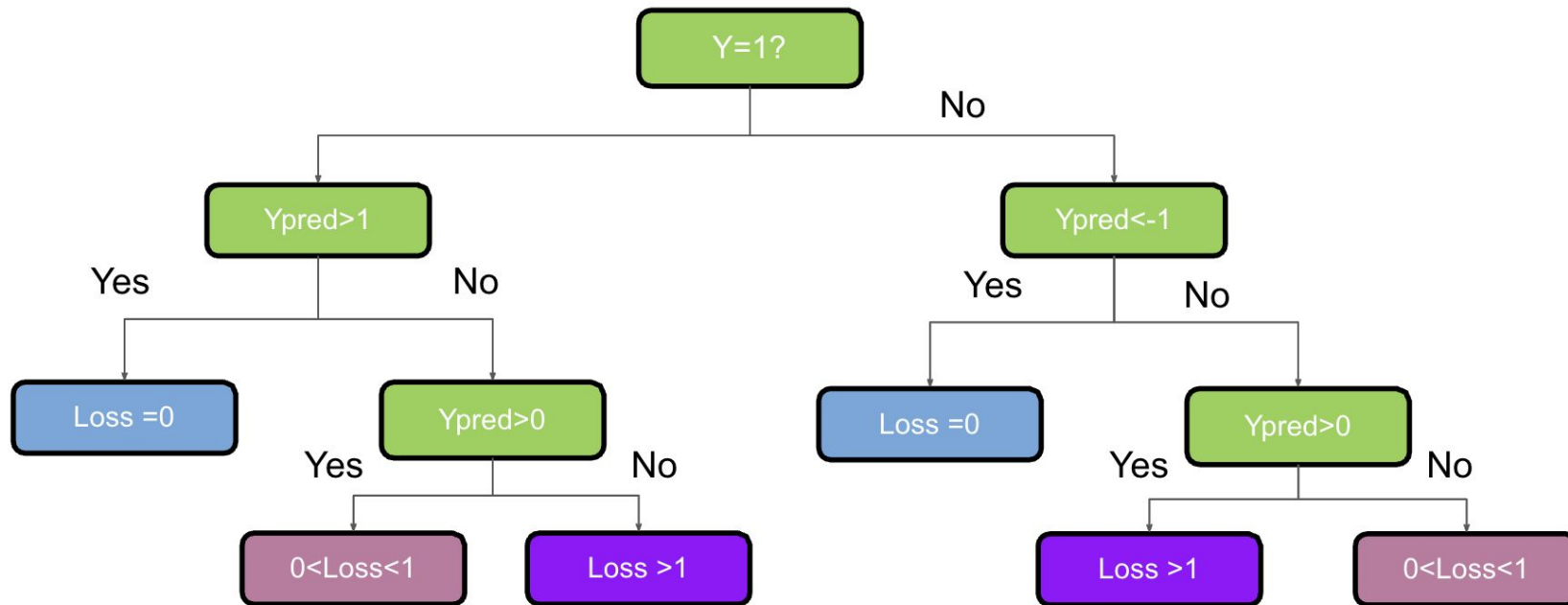
SVM Error Function

$$Loss = \max(0, 1 - y_{pred} \cdot y)$$

$y = \pm 1$

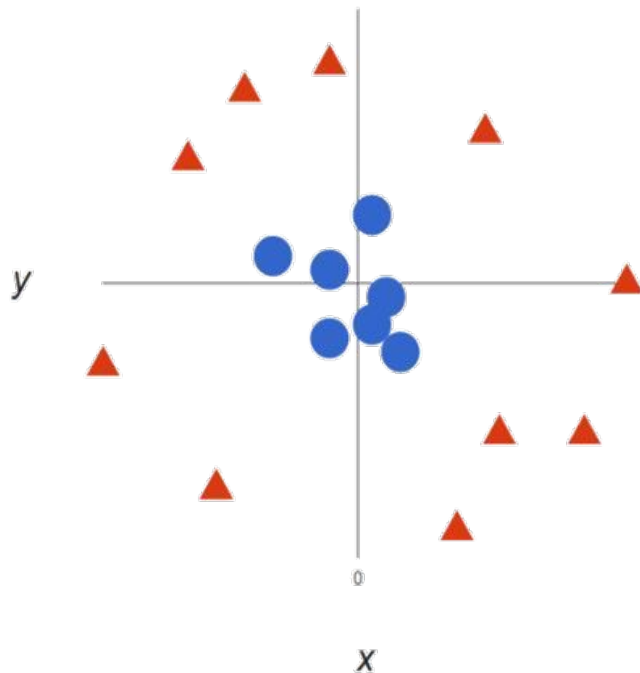


SVM Error Function



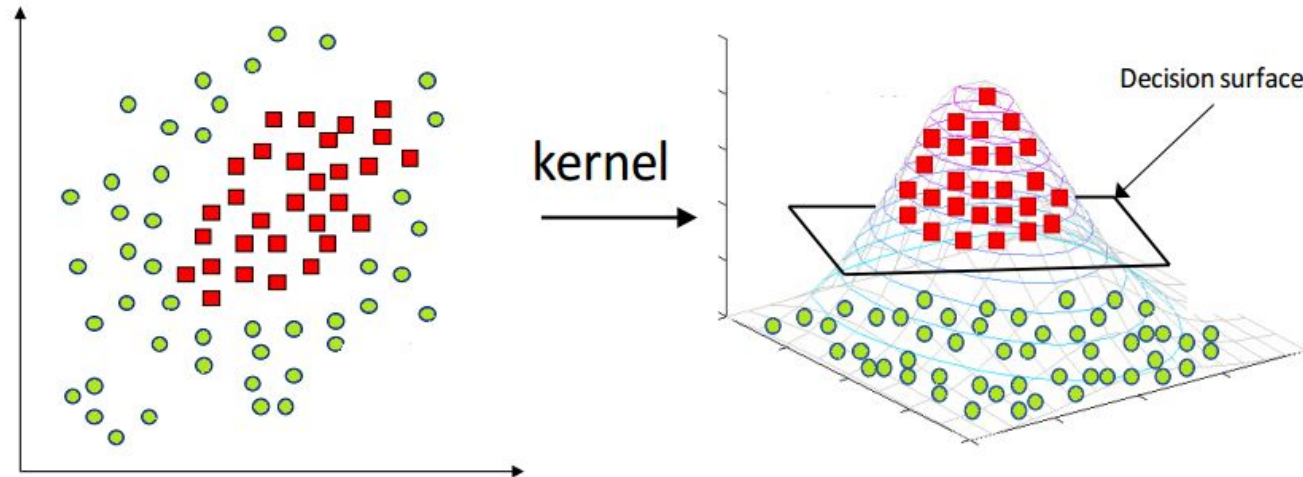
SVM for Classification

What if our dataset is not linearly separable?



SVM for Classification

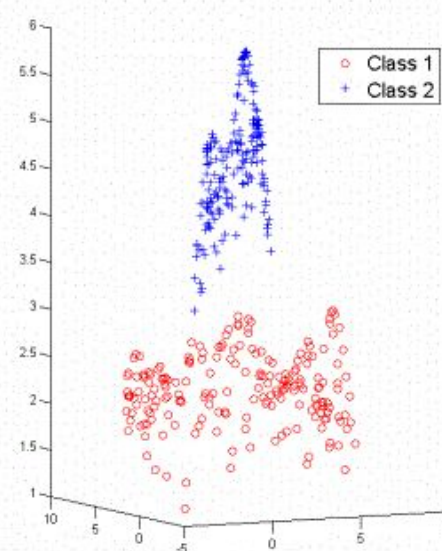
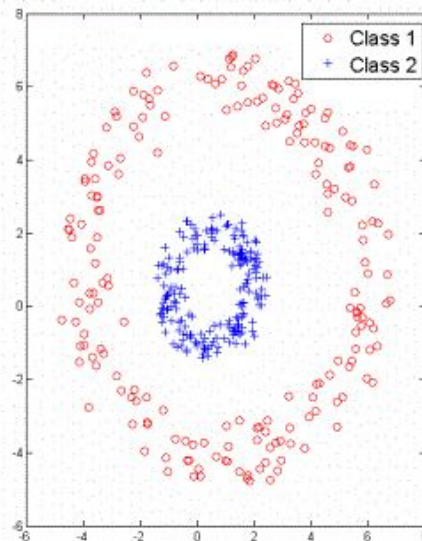
To solve this issue, we can increase the dimensionality of our dataset by adding a new feature. This would allow us to use a linear separator. This solution is called the “kernel trick”.



SVM for Classification

The kernel trick works by creating a new feature that is the function of the available features.

Given feature X and feature Y , we add a new feature Z where $X = X^2 + Y^2$

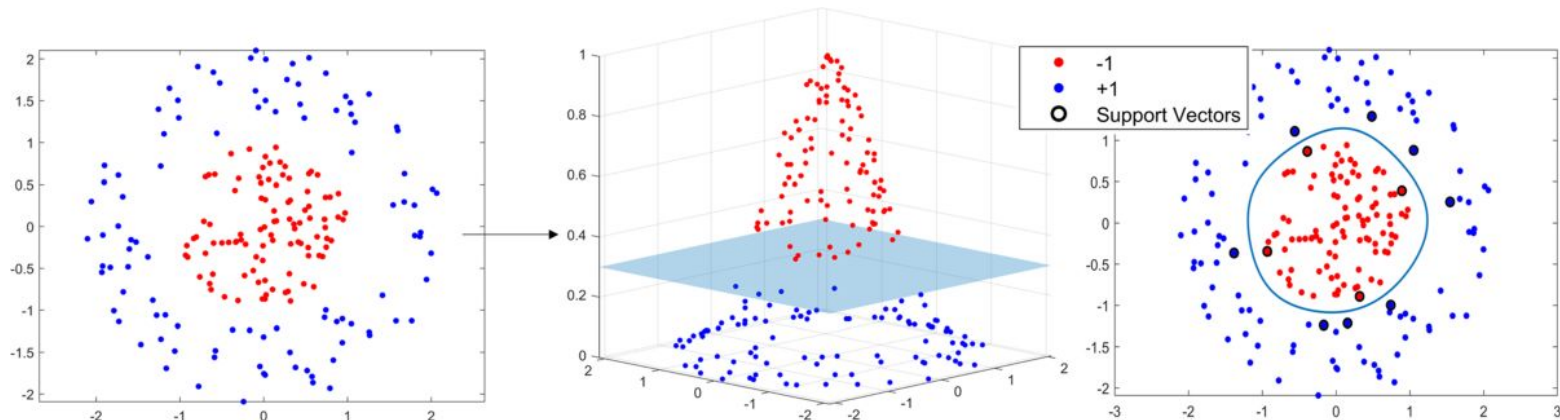


SVM for Classification

However, after training, we don't want to keep calculating a new feature for every input. Thus, we need to map the input back to the lower dimensional space.

Best Separating Hyperplane Equation:

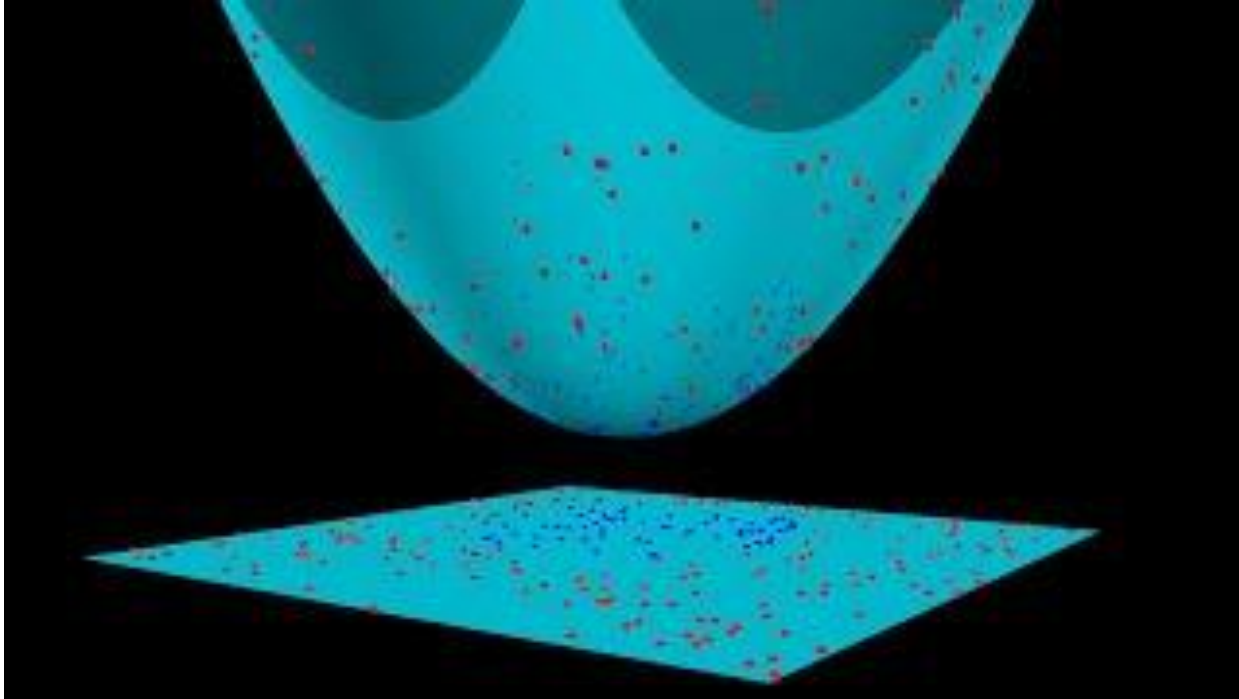
$Z = \text{constant} = X^2 + Y^2 \Leftrightarrow$ Equation of a circle in 2D space



SVM for Classification

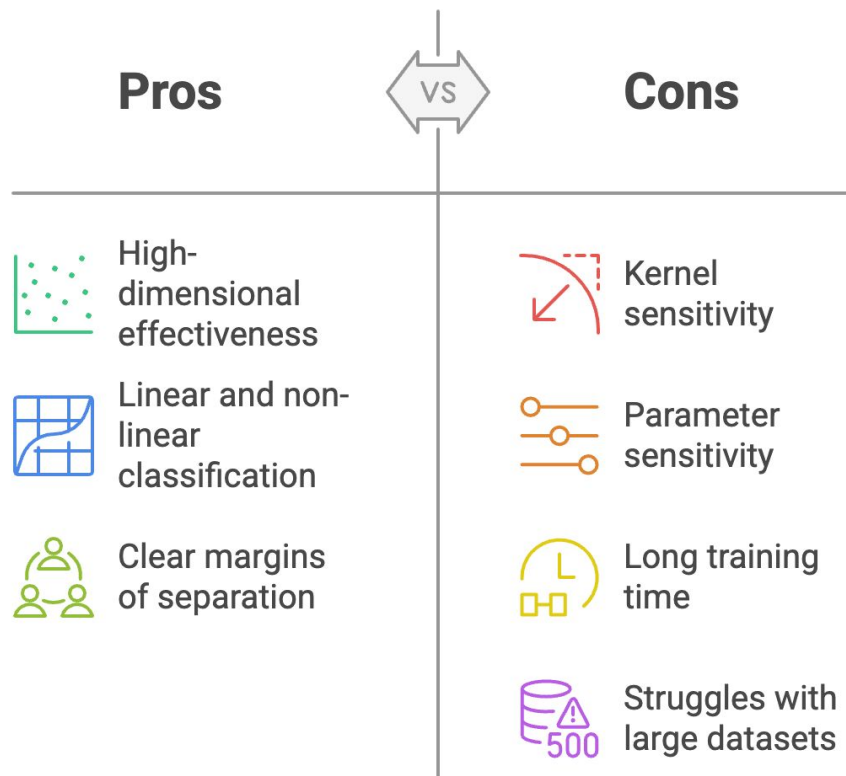


AMERICAN
UNIVERSITY
OF BEIRUT



SVM for classification

Support Vector Machines



Training SVM in lab ‘Classification Lab’

Training SVM Regressor in lab 'Regression Lab'

Decision Trees

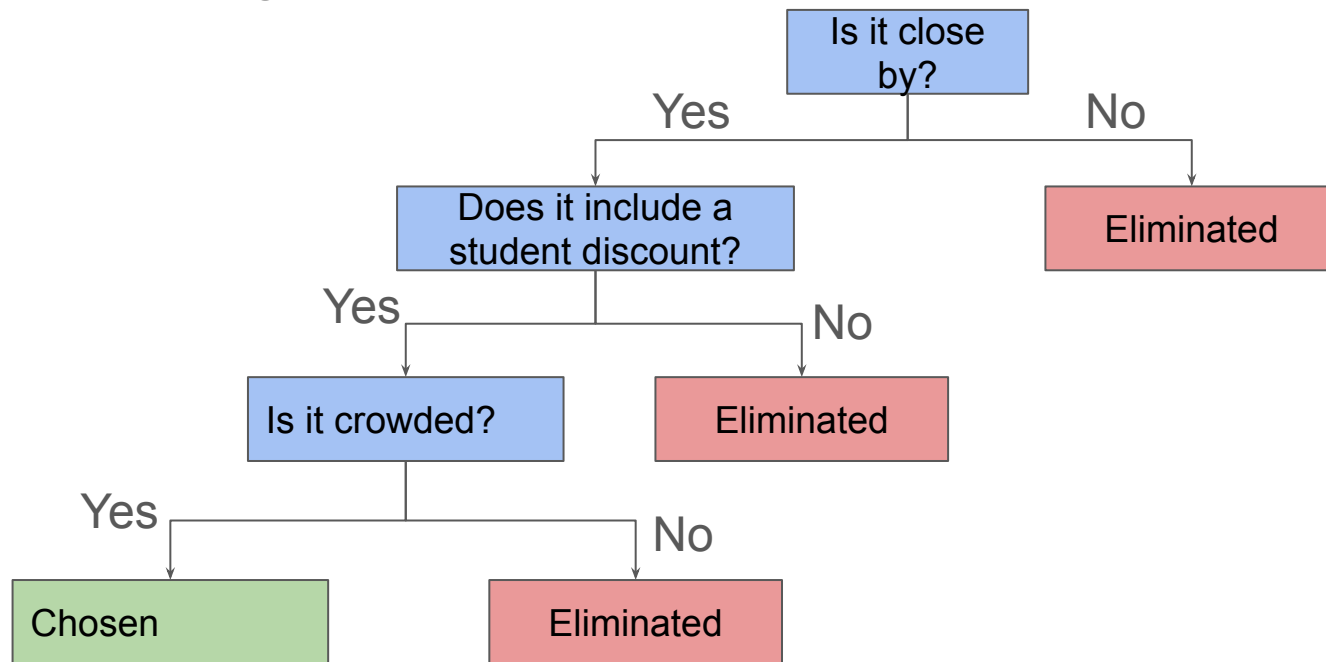
Decision Trees

Consider that you have a 2-hour break between classes, and you're looking for a place to eat.

What steps would you take to eliminate potential restaurants and eventually choose the best option?

Decision Trees

Your decision process might look like this:



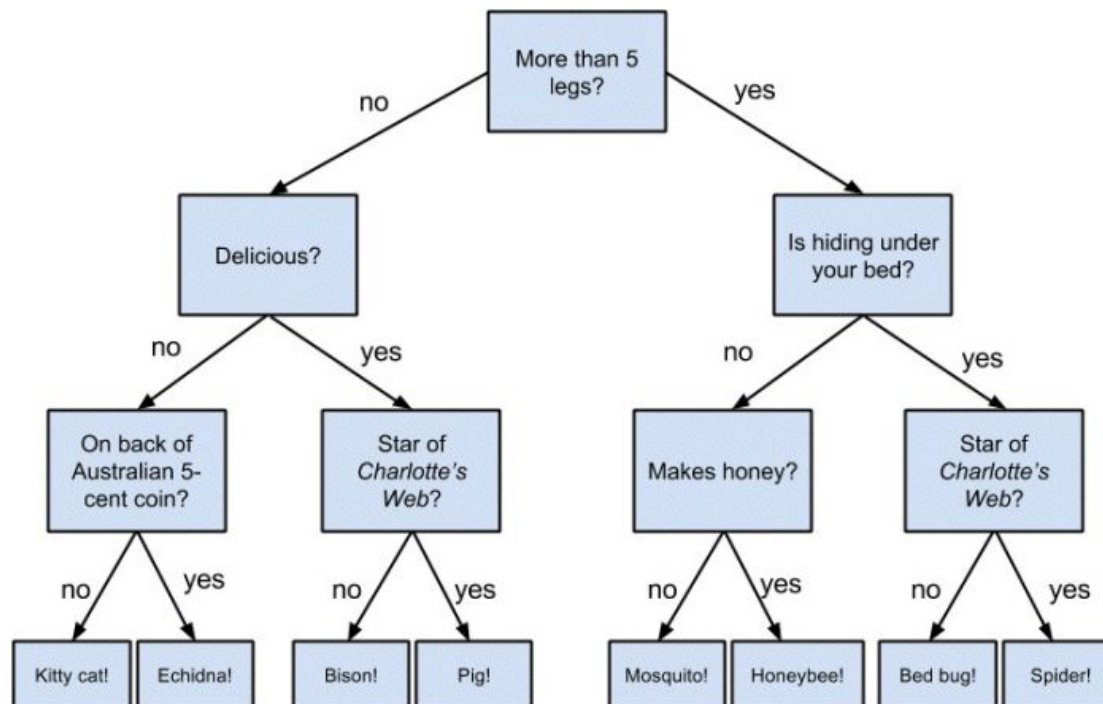
Decision Trees

Decision trees work in a similar manner where the algorithm tries to find the best set of rules that allows us to get to the final result.

It can be used to solve classification or regression problems

Decision Trees

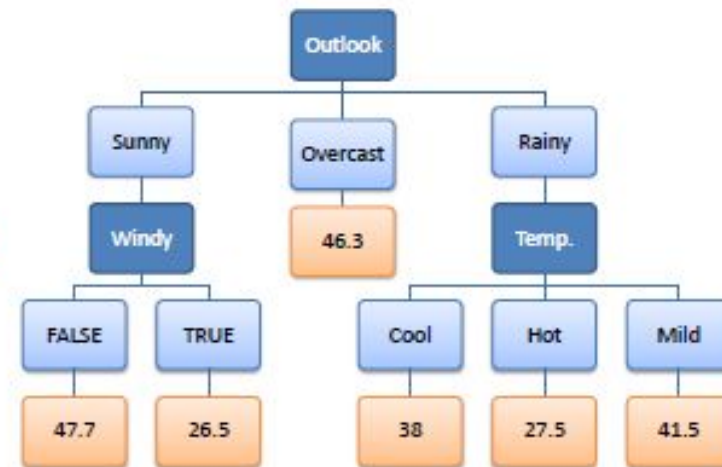
Classification example:



Decision Trees

Regression Example:

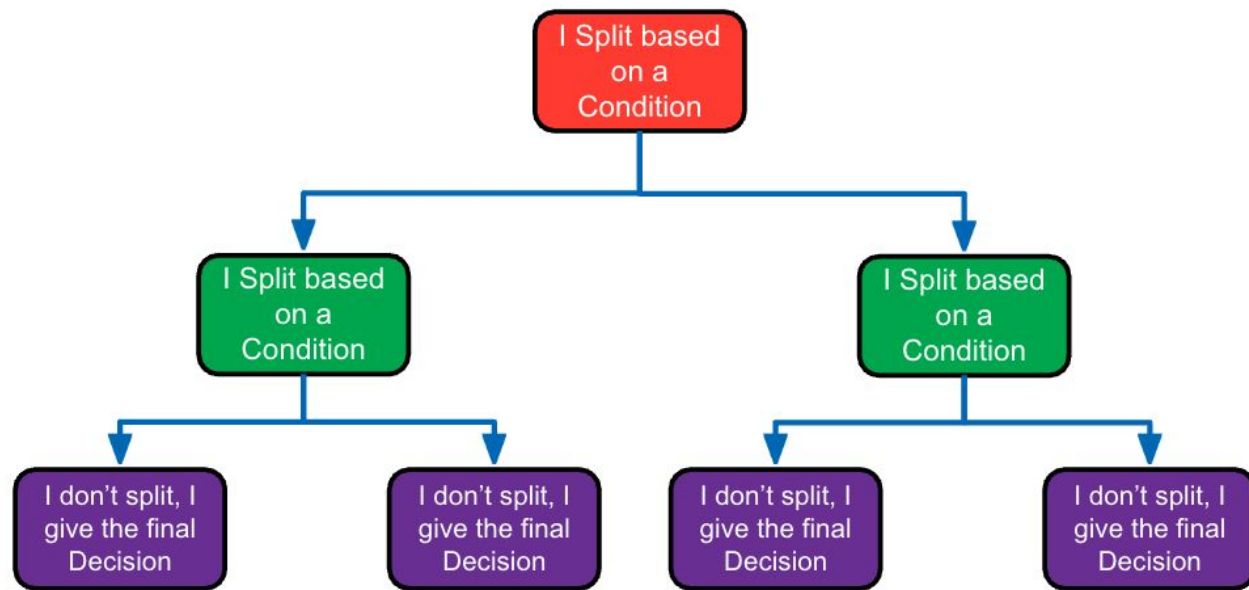
Predictors				Target
Outlook	Temp.	Humidity	Windy	Hours Played
Rainy	Hot	High	False	26
Rainy	Hot	High	True	30
Overcast	Hot	High	False	48
Sunny	Mild	High	False	46
Sunny	Cool	Normal	False	62
Sunny	Cool	Normal	True	23
Overcast	Cool	Normal	True	43
Rainy	Mild	High	False	36
Rainy	Cool	Normal	False	38
Sunny	Mild	Normal	False	48
Rainy	Mild	Normal	True	48
Overcast	Mild	High	True	62
Overcast	Hot	Normal	False	44
Sunny	Mild	High	True	30



Decision Trees

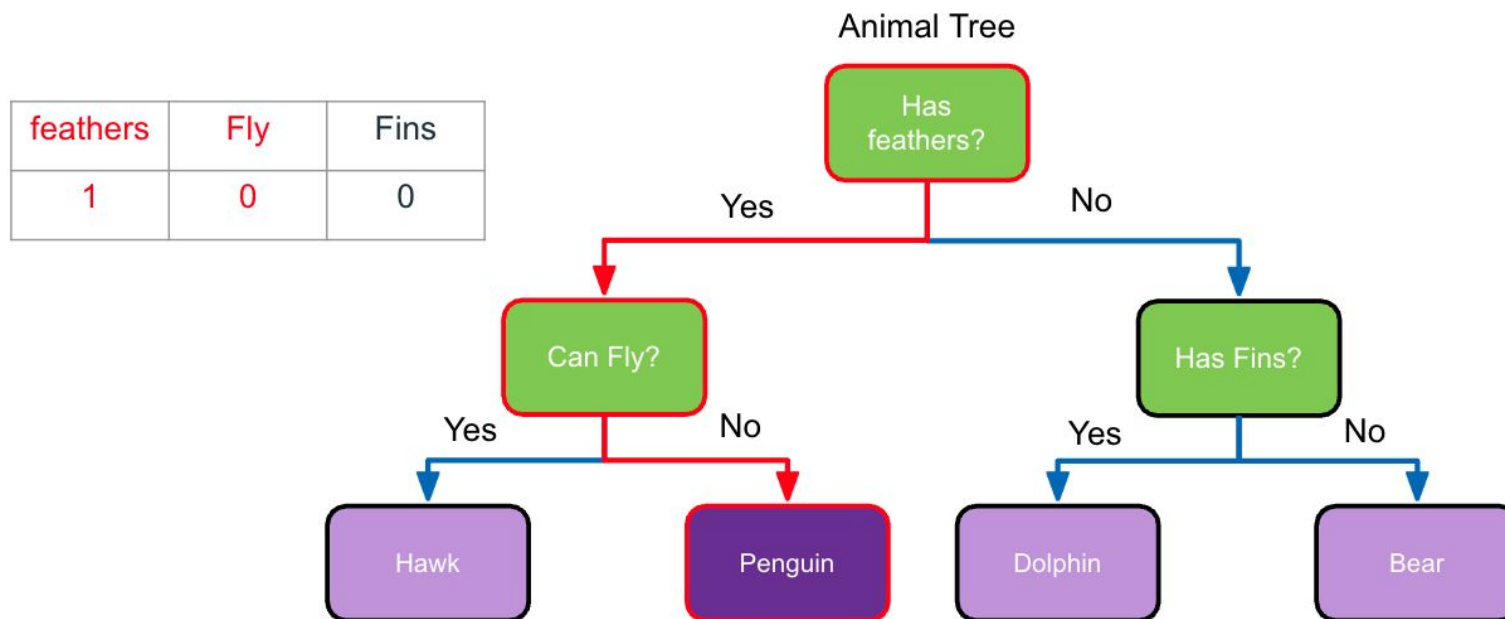
There are three types of nodes in a decision trees:

- Root Node
- Internal/Split Nodes
- Decision/Leaf Nodes



Decision Trees

Each decision node splits based on a feature.



Decision Trees

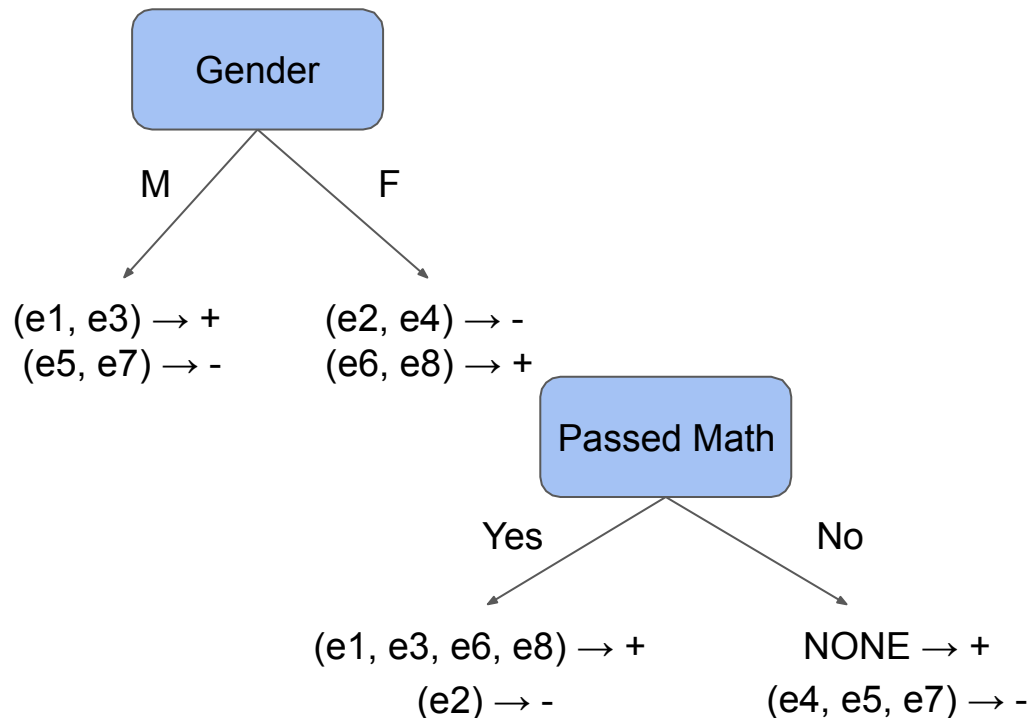
How does the algorithm choose which feature should be used at each position?

Each decision split should tell us something more about the branches we are heading towards.

For example, if we are trying to classify whether someone will pass or fail an academic year. Should we look at the gender or passed_math feature?

Decision Trees

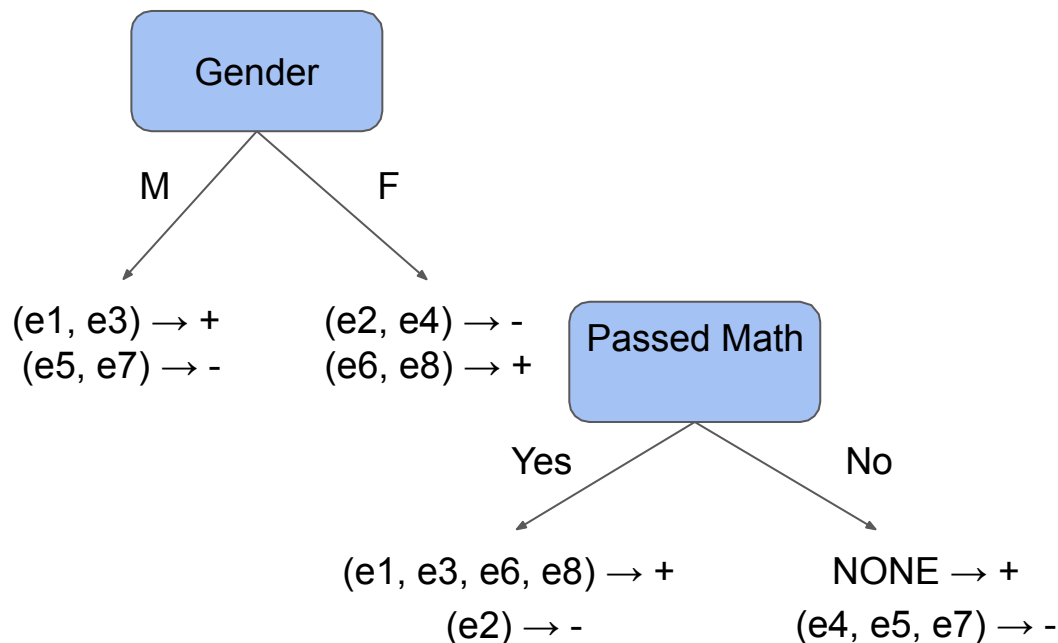
Let's test this theory.



Index	Gender	Passed Math	Passed
e1	M	1	1
e2	F	1	0
e3	M	1	1
e4	F	0	0
e5	M	0	0
e6	F	1	1
e7	M	0	0
e8	F	1	1

Decision Trees

Since the 'passed math' gives us more information towards reaching our final decision, then it is the better choice for the decision split.



Index	Gender	Passed Math	Passed
e1	M	1	1
e2	F	1	0
e3	M	1	1
e4	F	0	0
e5	M	0	0
e6	F	1	1
e7	M	0	0
e8	F	1	1

Decision Trees

How do we measure the amount of information that is inside a feature?

We can use probability to help us. If someone told you information that is common knowledge, you would not gain any information from it.

For example: If someone said “It’s very cold” during winter, you would not bat an eye.

If someone said “It’s very cold” during august, you would be shocked and might tell the people around you about what they said.

Decision Trees

Information gain is one way to measure the amount of information in a feature.
The higher the information gain, the higher the amount of information we have.

Thus, we calculate the information gain for every feature, and choose the one with the highest IG to use for splitting.

Information gain is based on **entropy**, which measure uncertainty.

Decision Trees

The greater our **uncertainty** about a situation, the lower its likelihood of occurring—and thus, the more informative the outcome.

Entropy $\longrightarrow H = E[I] = \sum_{i=1}^n -p_i \log_2(p_i)$

p_i is the probability of randomly selecting an example in class i

Decision Trees

For example, if we have three classes, we can calculate their entropy as follows:

Salary	Age	Purchase
20000	21	Yes
10000	45	No
60000	27	Yes
15000	31	No
30000	30	Maybe
12000	18	No
40000	40	Maybe
20000	20	Maybe

$$H(d) = -P_y \log_2(P_y) - P_n \log_2(P_n) - P_m \log_2(P_m)$$

$$H(d) = -2/8 \log_2(2/8) - 3/8 \log_2(3/8) - 3/8 \log_2(3/8)$$

$$H(d) = 1.56$$

Decision Trees

Now that we have the measure of uncertainty, we can calculate the information gain:

Number of instances
where the value of feature
A is v (for example in the
gender feature, the v
value could be male)

$$Gain(\textcolor{red}{S}, \textcolor{blue}{A}) = Entropy(\textcolor{red}{S}) - \sum_{v \in Values(A)} \frac{|\textcolor{green}{S}v|}{|\textcolor{red}{S}|} Entropy(\textcolor{green}{S}v)$$

Set of instances Feature

Training a Decision Tree

Let's take a step back and list out the training process:

1. We start at the root node, considering the entire training set.
2. Calculate information gain for all features, and choose the one with the highest information gain as the root node.
3. Recursively construct each subtree using the subset of the training set that remained down this path
4. Make sure that a feature is not repeated in one path.

Decision Trees

Another method of calculating the amount of information in a given feature is by measuring its **Gini Index**.

Unlike information gain, the gini index measures the amount of certainty in a piece of information. Thus, the lower the gini index, the higher the amount of information.

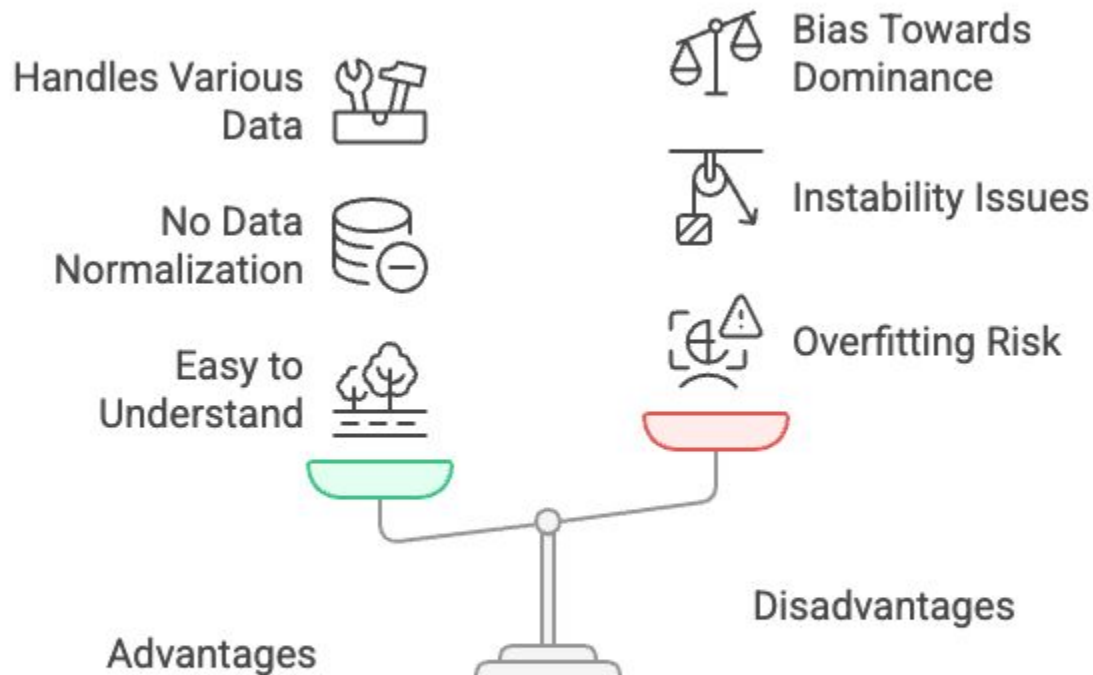
$$Gini\ Index = 1 - \sum_j p_j^2$$



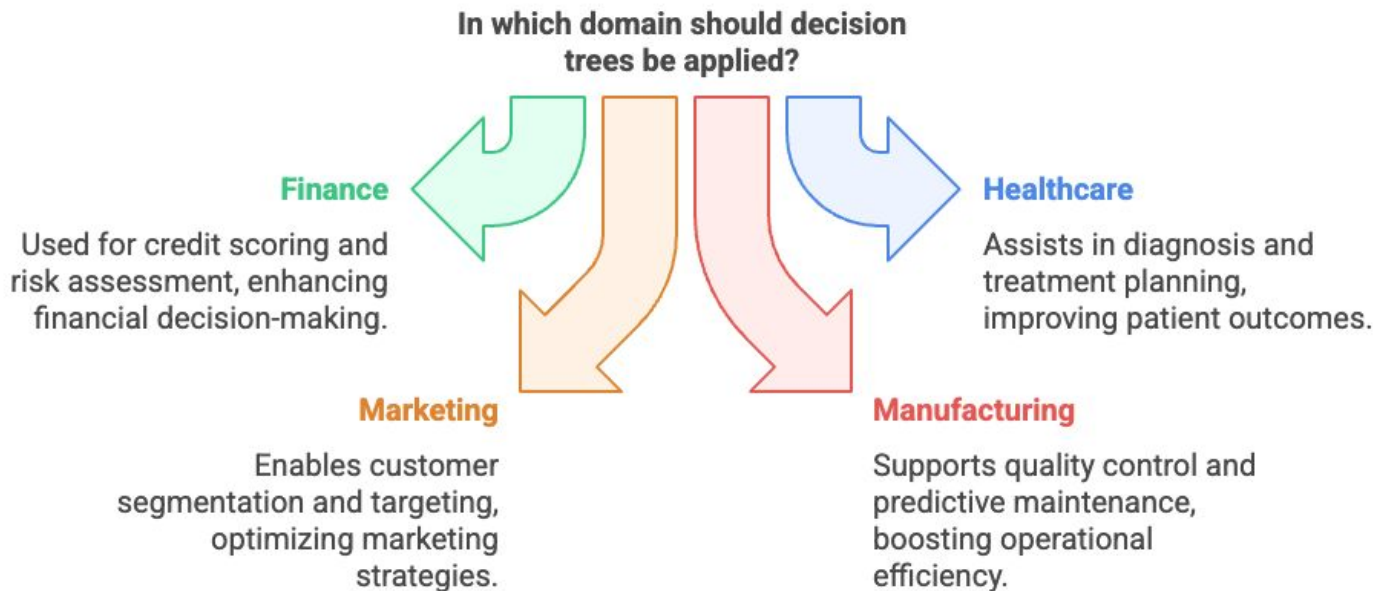
Decision Trees

Criterion	When to Use	Advantages	Disadvantages
Mini Index	- When you need a fast, computationally efficient measure (often used as a proxy for metrics like the Gini index).	- Simpler and faster to compute. - Less biased toward features with many unique values.	- May be less sensitive to subtle differences in class distributions.
Information Gain	- When a detailed evaluation of uncertainty reduction is important, especially for categorical splits (as in ID3/C4.5).	- Provides a clear measure of uncertainty reduction (entropy decrease). - Often yields more interpretable splits.	- Computationally more intensive. - Can favor attributes with many levels.

Decision Trees



Decision Trees



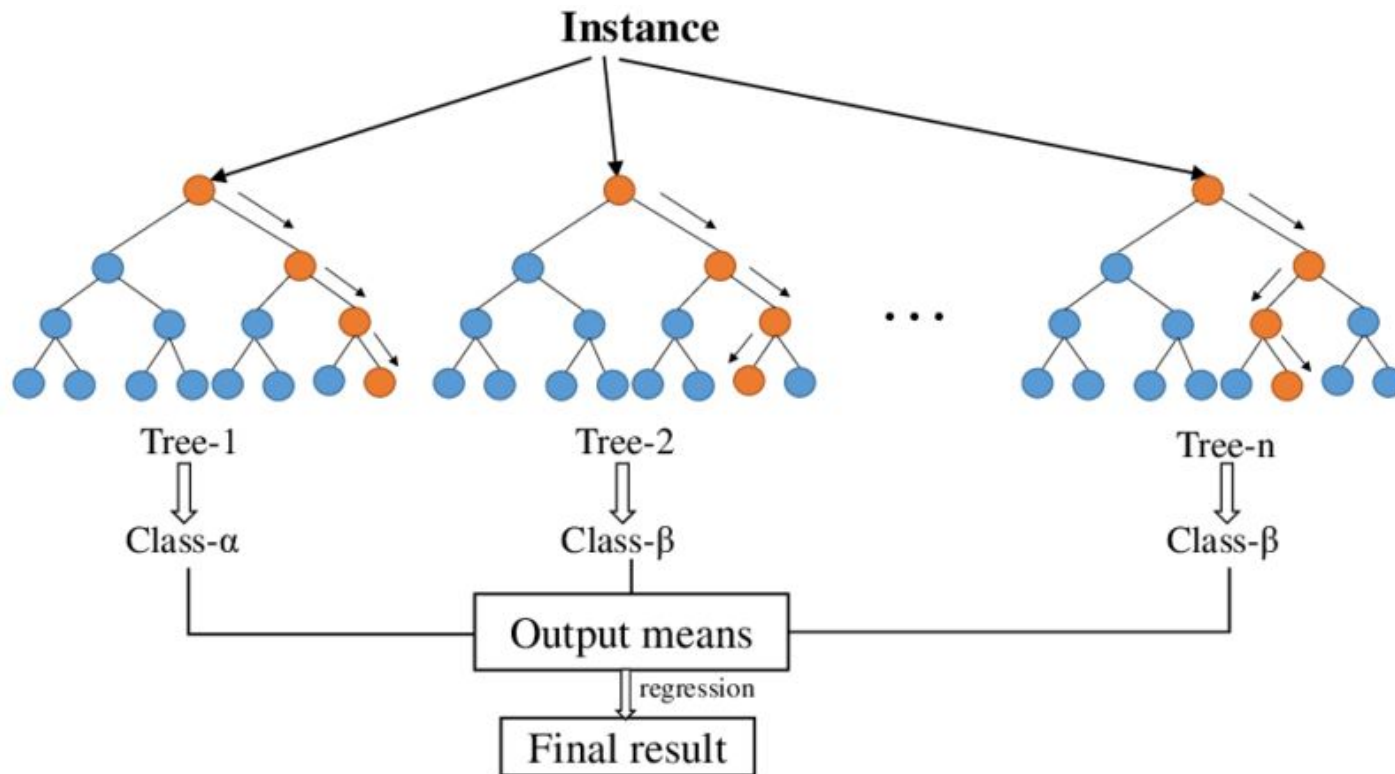
Decision Forests

Decision Forests

Decision trees are biased based on data splits because the splitting criteria (such as information gain) often favor features with many unique values or skewed distributions, which can lead to an imbalanced representation of feature importance.

To solve this issue, we create multiple decision trees and use a majority vote.

Decision Forests





Training Decision Trees in lab ‘Classification Algorithms’



Thank you